



操作系统实验报告

Lab 3: Programming with Threads

专 业： 人工智能

姓 名： 马承乾

学 号： 22920212204180

实验日期： 2023.12.28

指导教师： 曹冬林

目录

1	实验目的	4
1.1	实现事件栅栏原语并进行正确性测试;	4
1.2	实现闹钟原语并进行正确性测试;	4
1.3	用事件栅栏和闹钟原语来解决两个同步问题。	4
1.3.1	桥问题	4
1.3.2	电梯问题	4
2	实验要求	4
2.1	事件栅栏	4
2.2	实现闹钟原语	4
2.3	同步问题 1: 桥	5
2.4	同步问题 2: 电梯	5
3	实验流程 (介绍功能实现及测试)	6
3.1	。事件栅栏	6
3.1.1	分析	6
3.1.2	实现	6
3.1.3	测试	7
3.2	时钟	8
3.2.1	要求分析	8
3.2.2	实现	8
3.2.3	测试	10
3.3	桥	11
3.3.1	分析	11
3.3.2	实现	12
3.3.3	测试	17
3.3.4	实验结果分析	21
3.4	电梯	22
3.4.1	分析	22
3.4.2	实现	23
3.4.3	测试	29
4	实验总结	33
4.1	事件栅栏	33
4.2	闹钟原语	33
4.3	桥	34
4.4	电梯	34
5	实验心得	34
6	附录	34
6.1	threadtest.cc	34
6.2	Elevator.h	47
6.3	Elevator.cc	50
6.4	EventBarrier.h	62
6.5	EventBarrier.cc	63
6.6	Alarm.h	64

6.7	Alarm.cc.....	64
6.8	Bridge.h.....	66
6.9	Bridge.cc.....	68

1 实验目的

本次实验的目的在于掌握使用 nachos 中的线程编程解决较为复杂的并发问题, 具体分为三部分:

- 1.1 实现事件栅栏原语并进行正确性测试;
- 1.2 实现闹钟原语并进行正确性测试;
- 1.3 用事件栅栏和闹钟原语来解决两个同步问题。
 - 1.3.1 桥问题
 - 1.3.2 电梯问题

2 实验要求

2.1 事件栅栏

目标:

使用 Nachos 同步原语创建一个 EventBarrier 类, 允许一组线程以同步的方式等待事件并响应它。EventBarrier 结合了条件变量和信号量的有用属性。

功能:

- **void Wait()** — 等待直到事件被发出。如果已经处于发出状态, 则立即返回。
- **void Signal()** — 发出事件并阻塞, 直到所有等待此事件的线程都已响应。Signal()返回后, EventBarrier 回到未发出状态。
- **void Complete()** — 表明调用线程已完成对发出的事件的响应, 并阻塞, 直到所有等待此事件的其他线程也已响应。
- **int Waiters()** — 返回等待事件的线程数或尚未响应事件的线程数。

注意: 尽管它的名字是 **EventBarrier::Signal**, 但它更像是 **Condition::Broadcast** 而不是 **Condition::Signal**, 因为 **EventBarrier::Signal** 唤醒所有等待事件的线程。

实现: 您可以使用互斥锁、条件变量和/或信号量的任何组合来实现 EventBarrier, 但不要降低中断。

测试: 以您认为最合适的方式测试 EventBarrier 的实现。确保您的实现正确处理在 EventBarrier 对象处于发出状态时调用 Wait 的线程; 所有参与线程必须等待, 直到迟到的线程已响应事件并调用 Complete。此外, 您的实现应处理线程在从 Complete 返回后立即再次调用 Wait 的情况; 这些线程应该在 EventBarrier 再次发出信号之前在 Wait 中阻塞。

2.2 实现闹钟原语

目标:

实现一个 AlarmClock 类。线程通过调用 **Alarm::Pause(int howLong)** 来休眠一段时间。闹钟的实现可以使用模拟的计时器设备 (参见 **timer.h**)。当计时器中断发生时, **system.cc** 中的计时器中断处理程序必须唤醒所有在 **Alarm::Pause** 中等待的线程, 其时间间隔已经过期。被唤醒的线程应在至少等待了指定的间隔 (**howLong**) 后放入就绪队列。对于 **howLong**, 您可以使用任何方便的时间单位。

警告:

- 不要更改计时器硬件的行为来实现闹钟。您可以修改计时器中断处理程序, 但不要修改计时器类本身。创建新的 (模拟的) 硬件计时器设备也是不可接受的。
- 如果没有可运行的线程, Nachos 将退出, 即使有一个线程在等待闹钟。您必须找到一种方法来防止这种情况发生。这是一种罕见的情况, 其中“正确”的解决方案是修改

`machine` 子目录中的代码。在这种情况下，您可以修改机器，但这不是必需的。如果不修改机器，您需要想出一种方法来防止 Nachos 在有线程等待闹钟时退出。一种简单而不太美观的解决方法是在至少有一个线程等待闹钟时分支一个线程，该线程在一个紧密的循环中产生。

- 中断处理程序永远不应该休眠。考虑这个约束对您的同步方案的影响。此外，您应该设计您的代码以最小化中断时间的工作量。
- Boxed Nachos 只有在使用 `-rs` 选项时才会传递计时器中断。您可以使用 `-rs` 选项，但最好调整初始化代码，使其具有在有或没有 `-rs` 的情况下都有正确行为的正确行为。

2.3 同步问题 1：桥

要求：MTA 雇佣您负责在公共公路上的一座狭窄的轻型桥上同步交通。交通只能单向通过桥，如果桥上同时有超过 3 辆车，桥将无法承受其重量。在这个系统中，每辆车由一个线程表示，当车到达桥时，它会执行以下过程：

```
OneVehicle(int direc) {
    ArriveBridge(direc);
    CrossBridge(direc);
    ExitBridge(direc);
}
```

代码 1

在上述代码中，`direc` 是 0 或 1：它表示车辆将横穿桥的方向。编写 `ArriveBridge` 和 `ExitBridge` 过程（`CrossBridge` 过程应仅打印调试消息），使用您选择的任何涉及互斥锁、条件变量和/或信号量的同步方案。`ArriveBridge` 必须在车辆安全横穿桥的给定方向之前不返回（它必须保证不会发生迎头相撞或桥梁坍塌）。`ExitBridge` 被调用以指示调用者已经完成穿越桥梁，并且应该采取措施让其他车辆通过桥梁。为了测试您的程序，创建一些车辆，它们以不同的方向重复使用桥梁：使用您的 Alarm 时钟实现来模拟车辆的非桥梁活动。

注意：您应该尝试使您的解决方案公平且无饥饿。为了理解这涉及到什么，尝试回答以下有关您的方案的问题：如果桥上有车正在以一个方向 0 同行，此时新来了一辆同为 0 方向的车，但是已经有另一辆车等待以相反的方向 1 横穿桥（来得更早），新到达的车辆会在另一侧等待的车辆之前横穿，之后横穿，还是无法确定？

2.4 同步问题 2：电梯

任务：实现一个楼层有 F 层的大楼的电梯控制器，使用线程和您选择的同步方案，包括互斥锁、条件变量和/或信号量。您可能会发现您已经实现的 EventBarrier 原语在这里非常有用。

第一部分：实现单个电梯的电梯控制器，包括 `elevator.h` 中定义的 Building 和 Elevator 方法，以及您的实现所需的任何新方法。您的解决方案应避免乘客饥饿以及所有明显的竞态条件，例如电梯在运行时门打开、乘客在门关闭时进入和退出等。您可以假设电梯的大小是无限的，即它可以容纳所有到达的乘客。当电梯停在一个楼层时，它应该等到所有离开的乘客都离开并且所有上车的乘客都上车。

第二部分：扩展第一部分的解决方案，以处理每次只能容纳 N 名乘客的电梯，其中 N 是一个编译时常量。在这种情况下，如果电梯已满，您的 Enter 原语应返回失败状态，如果乘客试图在电梯已满时上车。

（附加分：20 分）：将第二部分的解决方案扩展到模拟具有 K (>1) 部电梯的大楼。您需

要想出一种方案来决定哪一个电梯用于服务请求。警告：许多团队发现这个问题非常困难。请早做准备。在编码之前进行思考。考虑几种可能的解决方案框架，然后选择最佳的一个。追求简单而优雅的解决方案，即使在性能方面不太优越也是可以的。

3 实验流程（介绍功能实现及测试）

3.1 。事件栅栏

3.1.1 分析

目标：

使用 Nachos 同步原语创建一个 EventBarrier 类，允许一组线程以同步的方式等待事件并响应它。EventBarrier 结合了条件变量和信号量的有用属性。

功能：

Wait():

等待直到事件被发出。如果已经处于发出状态，则立即返回。

Signal():

发出事件并阻塞，直到所有等待此事件的线程都已响应。Signal()返回后，EventBarrier 回到未发出状态。

Complete():

表明调用线程已完成对发出的事件的响应，并阻塞，直到所有等待此事件的其他线程也已响应。

Waiters():

返回等待事件的线程数或尚未响应事件的线程数。

3.1.2 实现

Wait():

```
void EventBarrier::Wait()
{
    lock->Acquire();
    waiternum++;
    if (signaled)// 如果事件栅栏已经处于 Signaled 状态，直接返回
    {
        lock->Release();
        return;
    }
    event->Wait(lock);
    lock->Release();
}
```

代码 2

获取锁，增加等待线程数量。

如果事件栅栏已经处于 Signaled 状态，直接释放锁并返回；否则，等待条件变量 event 的唤醒。

释放锁。

Signal():

```
void EventBarrier::Signal()
{
    lock->Acquire();
```

```

    signaled = true;
    event->Broadcast(lock);
    comp->Wait(lock);
    signaled = false;
    lock->Release();
}

```

代码 3

获取锁，设置事件栅栏为 Signaled 状态，广播条件变量 event，并等待条件变量 comp 的唤醒。

释放锁。

 **Complete():**

```

void EventBarrier::Complete()
{
    lock->Acquire();
    waiternum--;
    if (waiternum == 0)
    {
        comp->Broadcast(lock); // 唤醒所有线程
    }
    else
    {
        comp->Wait(lock); // 当前线程休眠
    }
    lock->Release();
}

```

代码 4

获取锁，减少等待线程数量。

如果当前等待线程数量为零，则广播条件变量 comp；否则，等待条件变量 comp 的唤醒。

释放锁。

 **Waiters():**

```

int EventBarrier::Waiters()
{
    return waiternum;
}

```

代码 5

返回等待线程数量 waiternum。

3.1.3 测试

-m 表示模式，m 为几代表在 threadtest 中会进入对应的分支函数。在本实验中，m=3 代表着测试事件栅栏。

我用红色表示线程被阻塞，绿色表示线程被 signal 或完成，用三个线程测试如下：

```

[ai22920212204180@mc core threads]$ ./nachos -m 3
thread 0 complete
waiters:0
thread 1 complete
waiters:1
thread 2 signaled
thread 1 finished
waiters:0
thread 0 finished
waiters:0
No threads ready or runnable, and no pending interrupts.
Assuming the program completed.
Machine halting!

Ticks: total 20000230, idle 0, system 20000230, user 0
Disk I/O: reads 0, writes 0
Console I/O: reads 0, writes 0
Paging: faults 0
Network I/O: packets received 0, sent 0

Cleaning up...

```

图表 1

如图，线程 0 和 1 完成之后受到阻塞，直到线程 2 完成后才一一完成。

3.2 时钟

3.2.1 要求分析

实现一个 `AlarmClock` 类，使得线程能够通过调用 `Alarm::Pause(int howLong)` 方法在指定的时间后被唤醒。要求保持计时器硬件的行为不变，不修改计时器类本身，也不创建新的硬件计时器设备。

构造函数和析构函数：

- 构造函数：初始化 `Alarm` 对象的成员变量，如等待队列等。
- 析构函数：释放可能分配的资源，确保没有内存泄漏。

Pause 方法：

- 使线程休眠指定的时间 `howLong`。
- 在计时器中断处理程序中唤醒等待的线程，并将其放入就绪队列。
- 注意处理等待队列的同步，防止竞态条件。

计时器中断处理程序：

- 在计时器中断发生时被调用。
- 负责唤醒等待的线程，并将其放入就绪队列。

Wakeup 方法：

- 唤醒应该醒来的睡眠线程。

3.2.2 实现

记录等待时间和当前线程：

- 在 `Pause` 方法开始时，记录当前时间和当前线程。使用 `Nachos` 提供的 `stats->totalTicks` 来获取当前时间。

计算唤醒时间：

- 根据参数 `howLong` 计算出线程应该被唤醒的时间。可以使用计时器中断的 `ticks` 单位和当前时间进行计算。

加锁和插入等待队列：

- 使用互斥锁（如 `Lock` 类）确保多线程情况下的同步。
- 将当前线程插入等待队列，并设置唤醒时间。此时，如果计时器中断处理程序已经执行，可能已经有线程到达唤醒时间。

释放锁并休眠：

- 释放互斥锁，允许其他线程访问等待队列。
- 调用当前线程的 Sleep 方法，使其休眠等待唤醒。

被唤醒后的处理：

- 在 Wakeup 方法中，计时器中断处理程序将唤醒时间到了的线程。

```
void Alarm::Pause(int howLong) // howlong 单位为中断次数
{
    waiters++;
    Thread *loop_t;
    if (waiters == 1)
    {
        loop_t = new Thread("loop thread");
        loop_t->Fork(check, 0); // 使至少有一个线程在运行，防止系统终止
    }
    IntStatus oldLevel = interrupt->SetLevel(IntOff); // 必须关中断
    int Ticks = TimerTicks * howLong; // timerticks 默认 100
    queue->SortedInsert((void *)currentThread,
                       stats->totalTicks + Ticks); // 按结束时间依序插入
链表
    interrupt->Schedule(Continue, (int)this, Ticks, TimerInt); // 用这一
行在 howlong 以后唤醒 sleep 的进程
    currentThread->Sleep();
    (void)interrupt->SetLevel(oldLevel);
}
```

代码 6

以上的函数负责使线程 sleep 并且加入等待队列，当时间到达 stats->totalTicks + Ticks 之后，schedule 将该线程再次唤醒，并将(int)this 传给 Continue 函数作为参数，Continue 函数实现如下：

```
void Alarm::Continue(int arg){// interrupt 要用得的 handler
    Alarm* temp = (Alarm*)arg;
    temp->Wakeup();
}
```

代码 7

Continue 将 int 类型的指针（传入的是 this 指针）转换回 Alarm 类型指针，并且调用 Wakeup 函数。按照 interrupt.cc 文件中 CheckIfDue 函数中的内容可知，凡是会调用 Continue 函数，进而来到 Wakeup 函数的进程，都是已经到达醒来时间的，所以在 Wakeup 函数中直接 ReadyToRun 即可。

```
void Alarm::Wakeup()
{
    int duetime = -1;
```

```

Thread *thread = NULL;
IntStatus oldLevel = interrupt->SetLevel(IntOff); // 必须关中断
thread = (Thread *)queue->SortedRemove(&duetime);
// 把 duetime 改成被删除的那个元素的 key 值，该 key 值即插入时间，将该值作为
thread 的名字
waiters--;
scheduler->ReadyToRun(thread);
printf("%s wakeup!\n", thread->getName());
(void)interrupt->SetLevel(oldLevel);
}

```

代码 8

3.2.3 测试

-m 表示模式，m 为几代表在 threadtest 中会进入对应的分支函数。在本实验中，m=4 对应测试时钟。

用蓝色表示线程进入休眠，绿色表示线程醒来。

```

[ai22920212204180@mc core threads]$ ./nachos -m 4
set thread 0 at 70th tick, timeslide is 1
set thread 1 at 90th tick, timeslide is 2
set thread 2 at 100th tick, timeslide is 3
set thread 3 at 110th tick, timeslide is 4
set thread 4 at 120th tick, timeslide is 5
thread 0 wakeup!
thread 0 wake up at 190th tick

thread 1 wakeup!
thread 1 wake up at 300th tick

thread 2 wakeup!
thread 2 wake up at 410th tick

thread 3 wakeup!
thread 3 wake up at 520th tick

thread 4 wakeup!
thread 4 wake up at 630th tick

No threads ready or runnable, and no pending interrupts.
Assuming the program completed.
Machine halting!

Ticks: total 630, idle 0, system 630, user 0
Disk I/O: reads 0, writes 0
Console I/O: reads 0, writes 0
Paging: faults 0
Network I/O: packets received 0, sent 0
Cleaning up...

```

图表 2

观察上图可知，线程醒来的时间与我们预期的时间有一定差异，查阅源代码可知 nachos 中时间是由指令模拟的，唤醒线程时调用 Continue 函数、Wakeup 函数、ReadyToRun 函数等指令都需要花费一定时间，所以延迟情有可原，具体时间延迟如下，蓝色表示睡眠时间，绿色表示延迟时间。一共五个线程，1500 个单位时间中，一共延迟了 60 个单位时间，不到 5%，这是可以接受的延迟。

表格 1：10 – 200 单位时间

- 减少桥上的车辆数量。
- 唤醒等待的车辆，使用条件变量或信号量。

3.3.2 实现

针对该问题，可以提出 FCFS、强化 FCFS、红绿灯、强化红绿灯四种算法，四种算法使用预定义方法进行切换，对每种方法，可以通过命令行确定车辆数、两个方向车辆的比例，车辆数和比例使用命令行参数确定。

3.3.2.1 FCFS

1. 整体思路如下：

一辆车靠近桥边时，若桥上车辆数量不超过三辆、且桥上车辆的行驶方向与当前车辆相同，而且当前车辆之前没有其他车辆，则允许当前车辆上桥；否则，当前车辆需要等待。为了标记当前车辆之前有没有其他车辆，使用 `pre_is_wait` 变量；为了标记桥上的车辆数目，使用 `on_bridge_num` 变量，为了标记桥上当前行驶的车辆的的方向，使用 `current_direc` 变量。

2. ArriveBridge 函数：

- 当一辆车到达桥时，首先通过互斥锁确保对共享资源的互斥访问。
- 判断当前桥上车辆数是否小于三辆，且桥上车辆方向与当前车辆方向相同。
- 如果满足条件，车辆可以上桥；否则，车辆需要等待。

```
void Bridge::ArriveBridge(int direc)
{
    lock->Acquire();
    printf("\033[1;34m%s car arrived the bridge in %d direc\033[0m\n\n",
        currentThread->getName(), direc);
    if (on_bridge_num == 0)
    {
        current_direc = direc;
    }
    while (direc != current_direc || on_bridge_num >= 3 || pre_is_wait)
    {
        pre_is_wait = true; // 下一辆车的前面有车
        con->Wait(lock); // 等待合适的时机再开始工作
        pre_is_wait = false; // 轮到我了！
        if (on_bridge_num == 0)
        {
            current_direc = direc;
        }
    }
    on_bridge_num++;
    lock->Release();
}
```

代码 9

3. CrossBridge 函数：

- CrossBridge 函数仅打印车辆通过桥的信息，没有涉及同步问题。Pause 方法模拟车辆在桥上行驶的时间。

```
void Bridge::CrossBridge(int direc)
{
    printf("\033[1;32m%s car is crossing the bridge in %d\n",
    direc\033[0m\n\n",
        currentThread->getName(), direc);
    Alarm::instance->Pause(CROSS_BRIDGE_TIME); // 在桥上走 10s
}
```

代码 10

4. ExitBridge 函数:

- 当车辆离开桥时，首先通过互斥锁确保对共享资源的互斥访问。
- 减少桥上车辆数，并使用条件变量通知其他等待的车辆。

```
void Bridge::ExitBridge(int direc)
{
    lock->Acquire();
    on_bridge_num--;
    printf("\033[1;33m%s car leave the bridge\033[0m\n\n",
currentThread->getName());
    con->Broadcast(lock); // 把被阻塞的线程一一取出
    lock->Release();
}
```

代码 11

3.3.2.2 加强 FCFS 算法

1. 总体思路:

有车辆 A,B,C 想上桥，A 和 C 的方向是 0，B 的方向是 1，此时 A 在桥上，B 和 C 不在桥上，C 应该上桥，但由于前一辆车 B 还没有上桥，且 B 因为和 A 的方向不一样而不上桥，所以 C 也不能上桥，这相当于浪费了让 C 上桥的机会。因此我们删去 pre_is_wait，避免这种浪费情况。

2. 具体实现:

此时其余代码与 FCFS 相同，唯一不同的是在车辆到达桥梁时的判断:

```
void Bridge::ArriveBridge(int direc)
{
    lock->Acquire();
    printf("\033[1;34m%s car arrived the bridge in %d\n",
        currentThread->getName(), direc);
    if (on_bridge_num == 0)
    {
        current_dirac = direc;
    }
    while (direc != current_dirac || on_bridge_num >= 3)
```

```

{
    con->Wait(lock);
    if (on_bridge_num == 0)
    {
        current_direc = direc;
    }
}
on_bridge_num++;
lock->Release();
}

```

代码 12

3.3.2.3 红绿灯算法

1. 总体设计思路：

每隔 BASE_GREEN_LIGHT_TIME 就更换一次桥上的车辆行驶方向，将其中一个方向的红灯和绿灯互换，也就是允许与否的权限互换，考虑 BASE_GREEN_LIGHT_TIME 时间到了但桥上还有车，这时要使用黄灯，使黄灯亮着，只要黄灯亮着就不许其他车辆上车，直到桥上所有车辆过桥，再熄灭黄灯，并使车辆行驶方向互换。

使用 manager 线程一直调用 CheckIfDue 函数，保证红绿灯的运行。

需要实现的过程有到达、过桥、离开三个：

2. 车辆到达桥的过程 (ArriveBridge)：

- 在到达桥的时候，首先调用 `CheckIfDue` 方法检查是否到达了切换方向的时间，以确保车辆等待黄灯的时候能够正常切换方向。
- 获取锁，保证对共享资源的互斥访问。
- 打印到达桥的消息，包括车辆名字和方向。
- 进入循环：
 - 当方向不符合或者桥上车辆数已满 3 辆，或者黄灯亮着时，车辆等待。
 - 在等待队列中记录等待车辆数，等待通行信号。
 - 等待结束后，更新等待车辆数。

```

void Bridge::ArriveBridge(int direc)
{
    CheckIfDue();
    lock->Acquire();
    printf("\033[1;34m%s car arrived the bridge in %d direc\033[0m\n\n",
        currentThread->getName(), direc);
    while (direc != current_direc || on_bridge_num >= 3 ||
yellow_light_on)
    {
        direc_car_num[direc]++;
        direc_con[direc]->Wait(lock);
        direc_car_num[direc]--;
    }
    on_bridge_num++;
}

```

```
lock->Release();
}
```

代码 13

3. 车辆过桥的过程 (CrossBridge) :

- 打印消息表示车辆正在过桥。
- 使用 Alarm 类的 Pause 方法模拟车辆过桥的时间，即暂停当前线程。

```
void Bridge::CrossBridge(int direc)
{
    printf("\033[1;32m%s car is crossing the bridge in %d\n",
        direc, currentThread->getName());
    Alarm::instance->Pause(CROSS_BRIDGE_TIME);
}
```

代码 14

4. 车辆离开桥的过程 (ExitBridge) :

- 获取锁，确保对共享资源的互斥访问。
- 更新桥上车辆数，并打印车辆离开消息。
- 使用条件变量广播唤醒等待队列上的车辆。
- 释放锁，结束访问共享资源。
- 调用 CheckIfDue 方法检查是否到达了切换方向的时间。

```
void Bridge::ExitBridge(int direc)// 离开
{
    lock->Acquire();
    on_bridge_num--;
    printf("\033[1;33m%s car leave the bridge\n",
        currentThread->getName());
    direc_con[direc]->Broadcast(lock);
    lock->Release();
    CheckIfDue();
}
```

代码 15

5. 车辆离开桥的过程 (ExitBridge) :

`if (stats->totalTicks >= next_switch_time):` 检查当前模拟的总运行时间是否已经超过了下一次交通灯切换的时间。

如果上述条件成立，说明到了切换交通灯的时候：

- `yellow_light_on = true;` 将黄灯状态置为 true，表示不许新车上桥。

- `if (on_bridge_num == 0)`: 如果当前桥上没有车辆，表示是一个合适的切换时机。
- `yellow_light_on = false;`: 关闭黄灯。
- `switch_status();`: 调用切换方向的函数，实现红绿灯的状态切换。

```
void Bridge::CheckIfDue()
{
    lock->Acquire();
    if (stats->totalTicks >= next_switch_time)
    {
        yellow_light_on = true;
        if (on_bridge_num == 0)
        {
            yellow_light_on = false;
            switch_status(); // 切换方向
        }
    }
    lock->Release();
}
```

代码 16

3.3.2.4 改进红绿灯算法

1. 总体设计思路:

考虑以下情况:

0 方向车辆已经全部过桥，此时桥梁只允许 0 方向车辆过桥;

1 方向由许多车辆处于等待队列，准备上桥，然后由于红绿灯不允许 1 方向上桥，系统只能忙等。

为了避免这种情况，当某个方向已经全部过桥时切换红绿灯，允许另一个方向车辆通行。

我们使用数组 `direc_car_num` 记录两个方向来车数量，这个量只能减少;

使用数组 `direc_car_can_go` 记录还能行进的车数，这个量只能增加;

实现如下:

2. ArriveBridge 函数:

- 目的:
 - 控制车辆到达桥的逻辑。
- 细节:
 - 获取锁，确保对共享资源的独占访问。
 - 检查允许车辆继续的条件:

- 如果桥为空，允许车辆继续。
- 如果桥不为空，但车辆的方向与当前方向匹配，并且桥上车辆少于三辆，则允许车辆继续。
- 否则，车辆进入等待状态使用条件变量（direc_con），并释放锁。
- 如果车辆正在等待，它将等待直到满足继续的条件。
- 更新特定方向等待的车辆计数。
- 在继续之前释放锁。

```
void Bridge::ArriveBridge(int direc)
{
    lock->Acquire();
    printf("\033[1;34m%s car arrived the bridge in %d direc\033[0m\n\n",
        currentThread->getName(), direc);
    direc_car_num[direc]++;
    while (direc != current_direc || on_bridge_num >= 3 ||
yellow_light_on || (direc_car_can_go[direc] <= 0 && switchCount > 1))
    {
        CheckIfDue();
        direc_con[direc]->Wait(lock);
    }
    if (switchCount <= 1)
    {
        direc_car_num[direc]--;
    }
    direc_car_can_go[direc]--;
    on_bridge_num++;
    lock->Release();
}
```

代码 17

其余函数与红绿灯算法基本一致，不赘述。

3.3.3 测试

为方便起见，用蓝色表示到达桥，绿色表示过桥，黄色表示离开桥

-m 参数表示选择测试的模式，在本实验中为 5 时测试桥同步问题；

-d 参数表示方向为 0 的车所占百分比；

-c 参数表示车辆总数；

3.3.3.1 FCFS

🚦 当两方向的车辆各占一半时：

<pre> [ai22920212204180@mc core threads]\$./nachos -m 5 car 0 wakeup! car 1 wakeup! +++car 0 start at 2040 ticks in 1direction car 0 car arrived the bridge in 1 direc car 0 car is crossing the bridge in 1 direc +++car 1 start at 2070 ticks in 0direction car 1 car arrived the bridge in 0 direc car 2 wakeup! +++car 2 start at 2660 ticks in 0direction car 2 car arrived the bridge in 0 direc car 0 wakeup! car 0 car leave the bridge ---car 0 costs 1060 car 1 car is crossing the bridge in 0 direc car 2 car is crossing the bridge in 0 direc car 3 wakeup! +++car 3 start at 3170 ticks in 1direction car 3 car arrived the bridge in 1 direc car 4 wakeup! +++car 4 start at 3480 ticks in 1direction </pre>	<pre> car 99 car arrived the bridge in 1 direc car 94 wakeup! car 95 wakeup! car 94 car leave the bridge ---car 94 costs 3320 car 98 wakeup! car 95 car leave the bridge ---car 95 costs 3160 car 99 car is crossing the bridge in 1 direc car 98 car leave the bridge ---car 98 costs 1610 car 99 wakeup! car 99 car leave the bridge ---car 99 costs 1700 car 97 car is crossing the bridge in 0 direc car 97 wakeup! car 97 car leave the bridge ---car 97 costs 4090 -----avg turnaround time 2625.20----- </pre>
---	---

图表 3

当一个方向占 0.3，另一个方向占 0.7 时：

<pre> [ai22920212204180@mc core threads]\$./nachos -m 5 car 0 wakeup! car 1 wakeup! +++car 0 start at 2040 ticks in 1direction car 0 car arrived the bridge in 1 direc car 0 car is crossing the bridge in 1 direc +++car 1 start at 2070 ticks in 0direction car 1 car arrived the bridge in 0 direc car 2 wakeup! +++car 2 start at 2660 ticks in 1direction car 2 car arrived the bridge in 1 direc car 0 wakeup! car 0 car leave the bridge ---car 0 costs 1060 car 1 car is crossing the bridge in 0 direc car 3 wakeup! +++car 3 start at 3170 ticks in 1direction </pre>	<pre> +++car 99 start at 48480 ticks in 1direction car 99 car arrived the bridge in 1 direc car 99 car is crossing the bridge in 1 direc car 96 wakeup! car 98 wakeup! car 96 car leave the bridge ---car 96 costs 2990 car 99 wakeup! car 98 car leave the bridge ---car 98 costs 2030 car 99 car leave the bridge ---car 99 costs 1110 -----avg turnaround time 2841.80----- </pre>
--	---

图表 4

3.3.3.2 强化 FCFS 算法

当左右方向车辆各占一半时：

<pre>[ai22920212204180@mc core threads]\$./nachos -m 5 car 0 wakeup! car 1 wakeup! +++car 0 start at 2040 ticks in 1direction car 0 car arrived the bridge in 1 direc car 0 car is crossing the bridge in 1 direc +++car 1 start at 2070 ticks in 0direction car 1 car arrived the bridge in 0 direc car 2 wakeup! +++car 2 start at 2660 ticks in 0direction car 2 car arrived the bridge in 0 direc car 0 wakeup! car 0 car leave the bridge ---car 0 costs 1060 car 1 car is crossing the bridge in 0 direc car 2 car is crossing the bridge in 0 direc car 3 wakeup! +++car 3 start at 3170 ticks in 1direction car 3 car arrived the bridge in 1 direc car 4 wakeup! +++car 4 start at 3480 ticks in 1direction car 4 car arrived the bridge in 1 direc car 5 wakeup! +++car 5 start at 3990 ticks in 0direction</pre>	<pre>car 97 wakeup! car 97 car leave the bridge ---car 97 costs 1500 car 94 car is crossing the bridge in 1 direc car 95 car is crossing the bridge in 1 direc car 98 car is crossing the bridge in 1 direc car 94 wakeup! car 95 wakeup! car 94 car leave the bridge car 98 wakeup! ---car 94 costs 4000 car 95 car leave the bridge ---car 95 costs 3840 car 99 car is crossing the bridge in 1 direc car 98 car leave the bridge ---car 98 costs 2210 car 99 wakeup! car 99 car leave the bridge ---car 99 costs 2370 -----avg turnaround time 3131.80-----</pre>
--	---

图表 5

当一个方向占 0.3，另一个方向占 0.7 时：

<pre>[ai22920212204180@mc core threads]\$./nachos -m 5 car 0 wakeup! car 1 wakeup! +++car 0 start at 2040 ticks in 1direction car 0 car arrived the bridge in 1 direc car 0 car is crossing the bridge in 1 direc +++car 1 start at 2070 ticks in 0direction car 1 car arrived the bridge in 0 direc car 2 wakeup! +++car 2 start at 2660 ticks in 1direction car 2 car arrived the bridge in 1 direc car 2 car is crossing the bridge in 1 direc car 0 wakeup! car 0 car leave the bridge ---car 0 costs 1060 car 3 wakeup! +++car 3 start at 3170 ticks in 1direction</pre>	<pre>car 98 car leave the bridge ---car 98 costs 1060 car 99 wakeup! car 99 car leave the bridge ---car 99 costs 1060 car 87 car is crossing the bridge in 0 direc car 97 car is crossing the bridge in 0 direc car 87 wakeup! car 97 wakeup! car 87 car leave the bridge ---car 87 costs 8750 car 97 car leave the bridge ---car 97 costs 3500 -----avg turnaround time 2636.70-----</pre>
--	--

图表 6

3.3.3.3 红绿灯算法

当左右方向车辆各占一半时：

```

---car 87 costs 6640

+++car 99 start at 48560 ticks in 1direction
car 99 car arrived the bridge in 1 direc
car 96 wakeup!
car 97 wakeup!
car 96 car leave the bridge

---car 96 costs 2900

car 97 car leave the bridge

---car 97 costs 2270

now switch the direc to 1
next switch at 63410 ticks
0 wait cars 0 1 wait cars 8
car 90 car is crossing the bridge in 1 direc
car 89 car is crossing the bridge in 1 direc
car 91 car is crossing the bridge in 1 direc

car 94 car leave the bridge

---car 94 costs 10060

car 95 wakeup!
car 98 car is crossing the bridge in 1 direc
car 99 car is crossing the bridge in 1 direc
car 95 car leave the bridge

---car 95 costs 9890

car 98 wakeup!
car 99 wakeup!
car 98 car leave the bridge

---car 98 costs 9390

car 99 car leave the bridge

---car 99 costs 8480

-----avg turnaround time 5839.80-----

```

图表 7

当一个方向占 0.3，另一个方向占 0.7 时：

```

[ai22920212204180@mc core threads]$ ./nuchos -m 5
now switch the direc to 0
next switch at 11040 ticks
0 wait cars 0 1 wait cars 0
car 0 wakeup!
car 1 wakeup!
+++car 0 start at 2120 ticks in 1direction
car 0 car arrived the bridge in 1 direc
+++car 1 start at 2160 ticks in 0direction
car 1 car arrived the bridge in 0 direc
car 1 car is crossing the bridge in 0 direc
car 2 wakeup!
+++car 2 start at 2730 ticks in 1direction
car 2 car arrived the bridge in 1 direc
car 1 wakeup!
car 3 wakeup!
car 1 car leave the bridge

---car 1 costs 1130

+++car 3 start at 3310 ticks in 1direction
car 95 car leave the bridge

---car 95 costs 16050

car 96 wakeup!
car 98 car is crossing the bridge in 1 direc
car 99 car is crossing the bridge in 1 direc
car 96 car leave the bridge

---car 96 costs 15610

car 98 wakeup!
car 99 wakeup!
car 98 car leave the bridge

---car 98 costs 15560

car 99 car leave the bridge

now switch the direc to 0
next switch at 73240 ticks
0 wait cars 0 1 wait cars 0
---car 99 costs 14760

-----avg turnaround time 10030.30-----

```

图表 8

3.3.3.4 强化红绿灯算法

考虑到在红绿灯算法中，等待

当左右方向车辆各占一半时：

```
[ai22920212204180@mc core threads]$ ./nachos -m 5
now switch the direc to 0
0 cars to next switch
0 come cars 0 1 come cars 0car 0 wakeup!
car 1 wakeup!
+++car 0 start at 2120 ticks in 1direction

car 0 car arrived the bridge in 1 direc
+++car 1 start at 2140 ticks in 0direction
car 1 car arrived the bridge in 0 direc
car 1 car is crossing the bridge in 0 direc
car 2 wakeup!
+++car 2 start at 2730 ticks in 0direction
car 2 car arrived the bridge in 0 direc
car 2 car is crossing the bridge in 0 direc
car 1 wakeup!
car 3 wakeup!

car 96 car is crossing the bridge in 0 direc
car 96 wakeup!
car 96 car leave the bridge
now switch the direc to 1
2 cars to next switch
0 come cars 0 1 come cars 2---car 96 costs 5430
car 98 car is crossing the bridge in 1 direc
car 99 car is crossing the bridge in 1 direc
car 98 wakeup!
car 99 wakeup!
car 98 car leave the bridge
---car 98 costs 5530
car 99 car leave the bridge
now switch the direc to 0
0 cars to next switch
0 come cars 0 1 come cars 0---car 99 costs 4580
-----avg turnaround time 4573.90-----
```

图表 9

当一个方向占 0.3，另一个方向占 0.7 时：

```
[ai22920212204180@mc core threads]$ ./nachos -m 5
now switch the direc to 0
0 cars to next switch
0 come cars 0 1 come cars 0car 0 wakeup!
car 1 wakeup!
+++car 0 start at 2120 ticks in 1direction

car 0 car arrived the bridge in 1 direc
+++car 1 start at 2140 ticks in 0direction
car 1 car arrived the bridge in 0 direc
car 1 car is crossing the bridge in 0 direc
car 2 wakeup!
+++car 2 start at 2730 ticks in 1direction
car 2 car arrived the bridge in 1 direc
car 1 wakeup!
car 1 car leave the bridge
car 3 wakeup!
---car 1 costs 1080
+++car 3 start at 3270 ticks in 1direction

car 96 car is crossing the bridge in 1 direc
car 98 car is crossing the bridge in 1 direc
car 99 car is crossing the bridge in 1 direc
car 96 wakeup!
car 98 wakeup!
car 99 wakeup!
car 96 car leave the bridge
---car 96 costs 6290
car 98 car leave the bridge
---car 98 costs 5330
car 99 car leave the bridge
now switch the direc to 0
0 cars to next switch
0 come cars 0 1 come cars 0---car 99 costs 4500
-----avg turnaround time 5177.00-----
```

图表 10

3.3.4 实验结果分析

表格 5 两侧车辆比例相同

	10	100	1000
FCFS	2319.00	2625.20	3704.47
加强 FCFS	2361.00	3131.80	3491.10
红绿灯	4925.00	5839.80	9677.32
加强红绿灯	4883.00	4573.90	8084.58

表格 6 两侧车辆比例不同

	10	100	1000
FCFS	2106.00	2841.80	3212.78
加强 FCFS	1904.00	2636.70	3075.69
红绿灯	5823.00	10030.30	81634.60
加强红绿灯	5766.00	5177.00	7664.72

将四种算法在不同情况下所用时间分析如下：

3.3.4.1 从车数目角度分析可知：

从 10 辆车到 1000 辆车，模拟情况越来越接近真实情况，情况越来越复杂，所以每辆车所花费平均时间也越来越多。

3.3.4.2 从四种算法角度分析可知：

两个加强版本的算法在车辆数较少时，与未加强版本差不多；
但在车辆数较多，较为复杂的情况下，加强版本的平均时间更少；

3.3.4.3 从两侧车辆比例进行分析可知：

先来先服务算法更擅长处理两侧车辆比例不同的情况；

红绿灯算法更擅长处理两侧车辆比例相同的情况。究其原因，先来先服务算法不像红绿灯算法，会专门为某个方向行进的车辆划定一个服务时间；直到这个时间段用完才允许切换桥上车辆的方向切换；加强版红绿灯算法则避免了这点，所以性能上较接近 FCFS 算法，但由于在允许一个方向的车辆上桥时，另一个方向的车辆长期被阻塞，所以平均耗费时间很长。

3.3.4.4 根据四种算法对实验中的问题进行回答：

两种 FCFS 算法：

新来的车会在另一侧的车之后上桥；

两种红绿灯算法：

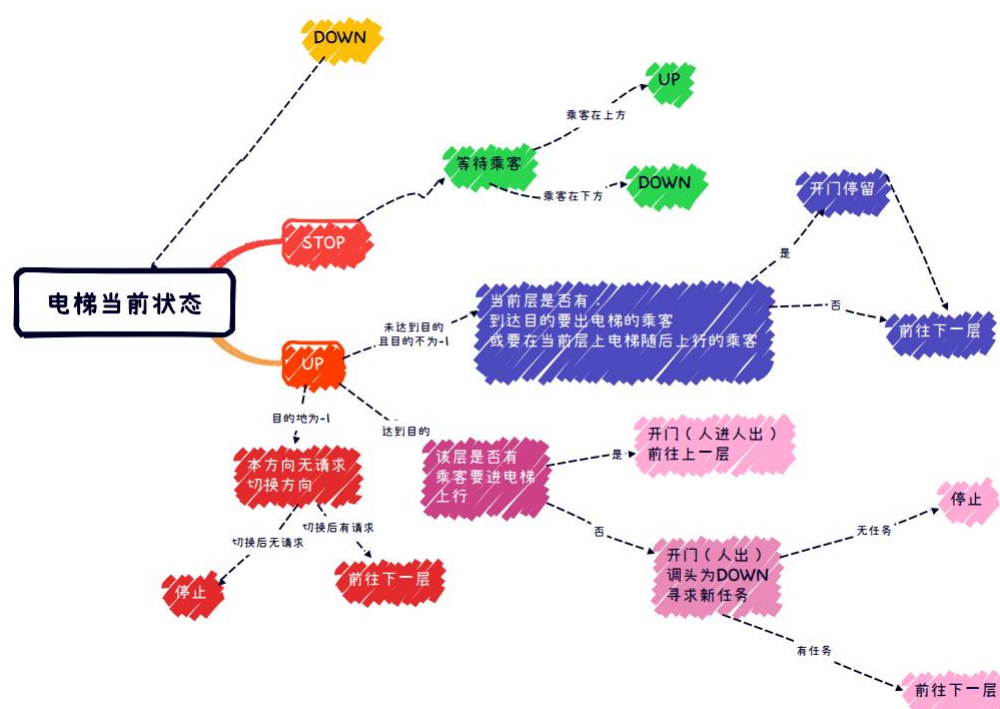
不一定，具体顺序要根据当前方向剩余的交通灯时间确定。

3.4 电梯

3.4.1 分析

设计电梯需要考虑以下三者：

3.4.1.1 总体设计逻辑



图表 11

3.4.1.2 电梯逻辑解释

一个电梯在不同楼层之间穿梭，需要考虑：

🚦 若电梯向 UP 方向前进，目的地为 100 层，途中 50 层遇见了向上和向下的 A 和 B，该

不该接 A 和 B?

A 顺路，接；B 不接。

✚ 若电梯向 UP 前进到达目的地 100 层，将乘客送出后，应该做什么？

当前层是否还有乘客希望继续 UP，若有则接上乘客并且继续 UP；

若无，则更换一个方向，查看是否有向下走的乘客，若有则向下走；

若两侧都无乘客，则停止。

✚ 注意：为什么若 100 层没有乘客希望上行就立刻调头呢？万一 105 层也有乘客呢？

我们在还没有到达目标 100 层的时候，每上一层楼就重新寻找目标，若更高层（高于 100）有目标则更新目标层，所以到达目标 100 层时，更高层不会有乘客在等待。

✚ 若电梯 UP 到 100 层，希望换方向，如何判断向下走可以完成任务？

从最底层向当前层找，若有一层乘客的起点位于 100 层以下，则可以向下。

✚ 若电梯到达目的地 100 层后，没有乘客，电梯怎么做？

答：停止不动。

3.4.1.3 建筑逻辑解释

在一个建筑中维护多个电梯，多个楼层，多个乘客，不同乘客处于不同楼层并抛出不同目的。

因此要考虑电梯的选择问题。

考虑以下问题：

✚ 当前在 100 层，上下都有电梯处于停止状态，选择哪一层的电梯？

选择最近的电梯。

3.4.2 实现

电梯调度策略，核心函数 ElevatorControl 设计如下：

3.4.2.1 电梯 ElevatorControl 中 STOP

设置 STOP，UP，DOWN 三种电梯状态，对一台电梯来说，若为 STOP 状态，则等待乘客发出请求，否则阻塞：

```
if (state == STOP)
{
    ElevatorLock->Acquire();
    printf("\033[1;32mElevator %d stop at %d floor with %d
riders\n\033[0m", ElevatorID, currentfloor, occupancy);
    if (dest_floor == -1)
    {
        HaveRequest->Wait(ElevatorLock); // 等人来
    }
    ElevatorLock->Release();
    printf("\033[1;32mElevator %d at %d floor with %d
riders\n\033[0m", ElevatorID, currentfloor, occupancy);
    dest_floor = GetLastRequestFloor(); // 等待电梯目标，没有目标的
话就会奔着-1层去，-1的情况会被特殊处理
    if (dest_floor > currentfloor)
        state = UP;
    else
        state = DOWN;
}
```

3.4.2.2 电梯 ElevatorControl 中 UP

若为 UP 状态，则一层层上行，上行中遇到同样上行的乘客，或者要出电梯的乘客，则开电梯门，然后再关上：

```
while (currentfloor != dest_floor && dest_floor != -1)
{
    no_requset_flag = false;
    if
(Building::instance->ElevatorUpBarrier[currentfloor]->Waiters() != 0
|| ElevatorOutBarrier[currentfloor]->Waiters() != 0)
    { // 我是上升的电梯，你是上升的乘客或者要出电梯的乘客
      // 根据常识，此时遇到下行的乘客，电梯不予理睬
        OpenDoors();
        CloseDoors();
    }
    VisitFloor(currentfloor + 1);
    dest_floor = GetLastRequestFloor();
    printf("\033[1;32mElevator %d at %d floor with %d
riders\n\033[0m",
        ElevatorID, currentfloor, occupancy);
}
```

代码 19

若当前目标是-1层，则首先换个方向 DOWN，查看有无乘客，DOWN 方向目标仍然是-1，则若直接停止：

```
if (dest_floor == -1) // 有多个电梯时，若请求被其他电梯满足，则
会被取消，故需在此处判断
{
    if (no_requset_flag) // 若在上一个循环中判断的另外一个方向也
无请求
    {
        state = STOP;
        continue;
    }
    state = DOWN; // 不知道去哪，换个方向吧
    no_requset_flag = true;
    continue;
}
```

代码 20

如果到达目的地了，则判断当前层有无乘客要上电梯并且向 UP 上行，若有则使其上电梯；若当前层没有要向 UP 上行的乘客，则换一个方向 DOWN 看看有无乘客下行：若 DOWN 方向有乘客则向 DOWN，若 DOWN 方向也无则 STOP：

```
if
(Building::instance->ElevatorUpBarrier[currentfloor]->Waiters() == 0)
    { // 到达目的地了，没乘客就调头，看另一个方向有无乘客
      state = DOWN; // 调头
    }
```



```

        OpenDoors();
        CloseDoors();
        dest_floor = GetLastRequestFloor();
        if (dest_floor == -1) // 若 DOWN 方向上也无新的请求则使电梯
停下
            state = STOP;
        else // 否则向 DOWN 方向运行
            VisitFloor(currentfloor - 1);
    }
    else // 有想从当前层向上的
    {
        OpenDoors();
        CloseDoors();
        VisitFloor(currentfloor + 1);
    }
}

```

代码 21

有则判断在所有楼层的新请求中，最高层的请求相对当前电梯位置是上还是下，上则继续上行，下则换个方向：

3.4.2.3 电梯 ElevatorControl 中 DOWN

DOWN 方向与 UP 对称。

3.4.2.4 电梯开门 OpenDoors

打开电梯门具体操作如下：

使用 alarm 类设置一个开门的时间片段，留出一定时间让乘客进出电梯；

假设当前电梯上行，若有乘客要出电梯，或者有乘客要上楼，则唤醒该类乘客，使其进出电梯。

用 ElevatorOutBarrier 数组表示每层要出电梯的乘客；

用 Building::instance->ElevatorUpBarrier 数据表示这栋楼中某一层要上行的乘客。

```

void Elevator::OpenDoors()
{
    Alarm::instance->Pause(OPEN_AND_CLOSE_DOOR);
    printf("\033[1;32mElevator %d open door at %d floor with %d
riders\n\033[0m", ElevatorID, currentfloor, occupancy);
    if (ElevatorOutBarrier[currentfloor]->Waiters() != 0)
    {
        ElevatorOutBarrier[currentfloor]->Signal();
    }
    if (state == UP)
    {
        if
(Building::instance->ElevatorUpBarrier[currentfloor]->Waiters() != 0)
        {
            Building::instance->ElevatorUpBarrier[currentfloor]->Signal(
);
        }
    }
}

```

```

    }
}
else if (state == DOWN)
{
    if
(Building::instance->ElevatorDownBarrier[currentfloor]->Waiters() != 0)
    {
        Building::instance->ElevatorDownBarrier[currentfloor]->Signal();
    }
}
}
}

```

代码 22

3.4.2.5 电梯关门 CloseDoors

关闭电梯门具体操作如下，输出调试信息，用 alarm 功能留出一段关门时间。

```

void Elevator::CloseDoors()
{
    printf("\033[1;32mElevator %d close door at %d floor with %d
riders\033[0m\n\n", ElevatorID, currentfloor, occupancy);
    Alarm::instance->Pause(OPEN_AND_CLOSE_DOOR);
}

```

代码 23

3.4.2.6 电梯停留 VisitFloor

输出调试信息，用 alarm 功能留出一段停留时间：

```

void Elevator::VisitFloor(int floor)
{
#ifdef BOUNDED_ELEVATOR
    ElevatorLock->Acquire();
    ElevatorNotFull->Broadcast(ElevatorLock);
    ElevatorLock->Release();
#endif
    Alarm::instance->Pause(ELEVATOR_MOVE_ONE_FLOOR);
    currentfloor = floor;
}

```

代码 24

3.4.2.7 乘客调度

下面以上行为例，说明具体上行过程中用到的函数：

3.4.2.8 乘客发出上行信号 CallUp

3.4.2.8.1 单电梯版本

向电梯发出请求：

```

void Building::CallUp(int fromFloor)
{
    elevator->ElevatorLock->Acquire();
    elevator->HaveRequest->Signal(elevator->ElevatorLock);
}

```

```

        elevator->ElevatorLock->Release();
    }

```

代码 25

3.4.2.8.2 多电梯版本

遍历所有电梯，选择最近的电梯进行调度：

```

void Building::CallUp(int fromFloor)
{
    BuildingLock->Acquire();
    int MinInterval = numFloors + 1;
    int eleNum = -1;
    for (int i = 0; i < numElevators; ++i)
    {
        if (elevator[i]->state == STOP && abs(elevator[i]->currentfloor
- fromFloor) < MinInterval)
        {
            MinInterval = abs(elevator[i]->currentfloor - fromFloor);
            eleNum = i;
        }
    }
    if (eleNum != -1)
    {
        elevator[eleNum]->HaveRequest->Signal(BuildingLock);
    }
    BuildingLock->Release();
}

```

代码 26

3.4.2.9 乘客发出请求后电梯还未到达当前楼层 Awaitup

3.4.2.9.1 单电梯版本

```

Elevator *Building::AwaitUp(int fromFloor)
{
    ElevatorUpBarrier[fromFloor]->Wait();
    return elevator;
}

```

代码 27

3.4.2.9.2 多电梯版本

若某个电梯正在当前层，且可进入，且同向（或处于停止态），则立即上电梯。

```

Elevator *Building::AwaitUp(int fromFloor)
{
    ElevatorUpBarrier[fromFloor]->Wait();
    BuildingLock->Acquire();
    Elevator *re;
    for (int i = 0; i < numElevators; ++i)
    {
        if (elevator[i]->currentfloor == fromFloor &&

```

```

        (elevator[i]->state == UP || elevator[i]->state == STOP) &&
elevator[i]->canEnter)
    {
        re = elevator[i];
        break;
    }
}
BuildingLock->Release();
return re;
}

```

代码 28

3.4.2.10 乘客进入电梯 Enter

3.4.2.10.1 无限容量

电梯中人数 occupancy 自增，建筑物对应楼层上行事件栅栏被唤醒。

```

bool Elevator::Enter()
{
    occupancy++;
    Alarm::instance->Pause(RIDER_ENTER_OUT);
    if (state == UP)
    {
        Building::instance->ElevatorUpBarrier[currentfloor]->Complete();
    }
    else if (state == DOWN)
    {
        Building::instance->ElevatorDownBarrier[currentfloor]->Complete(
);
    }
    return true;
}

```

代码 29

3.4.2.10.2 有限容量

此处人数超载，但仍然发出 Complete 信号，是为了让 Complete 数量和 Wait 数量一致。使用条件变量 ElevatorNotFull 来电梯是否“满”这个属性。

```

bool Elevator::Enter()
{
    occupancy++;
    if (occupancy > ELEVATOR_CAPACITY)
    {
        occupancy--;
        if (state == UP)
        {
            Building::instance->ElevatorUpBarrier[currentfloor]->Comple
e();
        }
    }
}

```

```

        else if (state == DOWN)
        {
            Building::instance->ElevatorDownBarrier[currentfloor]->Complete();
        }
        ElevatorLock->Acquire();
        ElevatorNotFull->Wait(ElevatorLock);
        ElevatorLock->Release();
        return false;
    }
}

```

代码 30

3.4.2.11 乘客到达某层但还没到目标层 RequestFloor

```

void Elevator::RequestFloor(int floor)
{
    ElevatorOutBarrier[floor]->Wait();
}

```

代码 31

3.4.2.12 乘客达到目标层并离开 Exit

```

void Elevator::Exit()
{
    Alarm::instance->Pause(RIDER_ENTER_OUT);
    occupancy--;
    ElevatorOutBarrier[currentfloor]->Complete();
}

```

代码 32

3.4.3 测试

🚦 -m 表示模式，m 为几代表在 threadtest 中会进入对应的分支函数，参数 b 代表电梯个数，参数 f 表示乘客数目，x 表示楼层数。在本实验中，6 对应测试电梯。

🚦 为了表示清晰，用蓝色表示乘客的起点、终点、时间信息；用绿色表示电梯活动；容量在 Elevator.h 的宏定义中进行修改。

3.4.3.1 单个电梯，容量无限情况，1000 个乘客，1000 层（测试鲁棒性）

```

[ai22920212204180@mc core threads]$ ./nachos -m 6 -f 1000 -x 1000
Elevator 0 stop at 0 floor with 0 riders
rider 0 wakeup!
rider 1 wakeup!
rider 2 wakeup!
rider 3 wakeup!
rider 4 wakeup!
rider 5 wakeup!
rider 6 wakeup!
rider 7 wakeup!
rider 8 wakeup!
rider 9 wakeup!
rider 10 wakeup!
rider 11 wakeup!
rider 12 wakeup!
rider 13 wakeup!
rider 14 wakeup!
rider 15 wakeup!
rider 16 wakeup!
rider 17 wakeup!
rider 18 wakeup!
rider 19 wakeup!
rider 20 wakeup!
Elevator 0 at 936 floor with 1 riders
elevator wakeup!
Elevator 0 at 937 floor with 1 riders
elevator wakeup!
Elevator 0 at 938 floor with 1 riders
elevator wakeup!
Elevator 0 at 939 floor with 1 riders
elevator wakeup!
Elevator 0 at 940 floor with 1 riders
elevator wakeup!
Elevator 0 at 941 floor with 1 riders
elevator wakeup!
Elevator 0 at 942 floor with 1 riders
elevator wakeup!
Elevator 0 open door at 942 floor with 1 riders
Rider 600 Exit
rider 600 wakeup!
Rider 600 finished
rider 600 costs 18447900
-----avg turnaround time 8647813.48-----
-----max turnaround time 18447900-----
-----min turnaround time 106630-----
Elevator 0 close door at 942 floor with 0 riders
elevator wakeup!
Elevator 0 at 942 floor with 0 riders
Elevator 0 stop at 942 floor with 0 riders

```

图表 12

算法执行成功，可知其鲁棒性值得相信。

3.4.3.2 单个电梯，容量 20，1000 个乘客，1000 层（测试鲁棒性）

```
[ai22920212204180@mc core threads]$ ./nachos -m 6 -f 1000 -x 1000
Elevator 0 stop at 0 floor with 0 riders
rider 0 wakeup!
rider 1 wakeup!
rider 2 wakeup!
rider 3 wakeup!
rider 4 wakeup!
rider 5 wakeup!
rider 6 wakeup!
rider 7 wakeup!
rider 8 wakeup!
rider 9 wakeup!
rider 10 wakeup!
rider 11 wakeup!
rider 12 wakeup!
rider 13 wakeup!
rider 14 wakeup!
rider 15 wakeup!
rider 16 wakeup!
rider 17 wakeup!
rider 18 wakeup!
rider 19 wakeup!
rider 20 wakeup!
rider 21 wakeup!
rider 22 wakeup!
rider 23 wakeup!
rider 24 wakeup!
rider 25 wakeup!

elevator wakeup!
elevator wakeup!
Elevator 0 at 4 floor with 1 riders
elevator wakeup!
Elevator 0 at 3 floor with 1 riders
elevator wakeup!
Elevator 0 at 2 floor with 1 riders
elevator wakeup!
Elevator 0 at 1 floor with 1 riders
elevator wakeup!
Elevator 0 at 0 floor with 1 riders
elevator wakeup!
Elevator 0 open door at 0 floor with 1 riders
Rider 599 Exit
rider 599 wakeup!
Rider 599 finished

rider 599 costs 121594980

-----avg turnaround time 54221579.85-----
-----max turnaround time 121594980-----
-----min turnaround time 107180-----

Elevator 0 close door at 0 floor with 0 riders

elevator wakeup!
Elevator 0 at 0 floor with 0 riders
Elevator 0 stop at 0 floor with 0 riders
```

图表 13

算法执行成功，验证了其鲁棒性。

3.4.3.3 单个电梯模式，容量 3，三个乘客，一共三层

首先设置容量，随后开始运行：

```
code > threads > C Elevator.h > ELEVATOR_CAPACITY
1  #ifndef ELEVATOR_H
2  #define ELEVATOR_H
3  #include "EventBarrier.h"
4  #include "system.h"
5  #include "synch.h"
6
7  #define SINGLE_ELEVATOR
8  // #define MULTIPLE_ELEVATOR // 单个或多个电梯
9  #define BOUNDED_ELEVATOR
10 // #define UNBOUNDED_ELEVATOR // 电梯容量是否有限
11 #define OPEN_AND_CLOSE_DOOR 20 // 电梯开关门的时间
12 #define RIDER_ENTER_OUT 10 // 一个乘客进出电梯的时间
13 #define ELEVATOR_MOVE_ONE_FLOOR 40 // 电梯移动一层楼的时间
14 #ifdef BOUNDED_ELEVATOR
15 #define ELEVATOR_CAPACITY 3 // 电梯容量
16 #endif
17 > enum ElevatorState...
23 void rider(int id, int srcFloor, int dstFloor);
24 > class Elevator...
61 > class Building...
97 #endif // ELEVATOR
98
```

图表 14

结果如下：

```

[ai22920212204180@ncore threads]$ ./nuchos -m 6 -f 3 -x 3
Elevator 0 stop at 0 floor with 0 riders
rider 0 wakeup!

rider 0 start at 220 ticks from 0 floor to 1 floor
Rider 0 CallUp from 0th floor
Rider 0 wait to go up from 0th floor

Elevator 0 at 0 floor with 0 riders
rider 1 wakeup!

rider 1 start at 360 ticks from 1 floor to 2 floor
Rider 1 CallUp from 1th floor
Rider 1 wait to go up from 1th floor

rider 2 wakeup!

rider 2 start at 440 ticks from 2 floor to 1 floor
Rider 2 CallDown from 2th floor
Rider 2 want to go down to 2th flooe

elevator wakeup!
Elevator 0 open door at 0 floor with 0 riders
Rider 0 Enter() elevator 0
rider 0 wakeup!
Rider 0 want to go to 1th floor
Elevator 0 close door at 0 floor with 1 riders

Elevator 0 open door at 2 floor with 1 riders
Rider 1 Exit
rider 1 wakeup!
Rider 1 finished

rider 1 costs 22690
Rider 2 Enter() elevator 0
rider 2 wakeup!
Rider 2 want to go to 1th floor
Elevator 0 close door at 2 floor with 1 riders

elevator wakeup!
elevator wakeup!
Elevator 0 at 1 floor with 1 riders
elevator wakeup!
Elevator 0 open door at 1 floor with 1 riders
Rider 2 Exit
rider 2 wakeup!
Rider 2 finished

rider 2 costs 32980

-----avg turnaround time 22720.00-----
-----max turnaround time 32980-----
-----min turnaround time 12490-----

```

图表 15

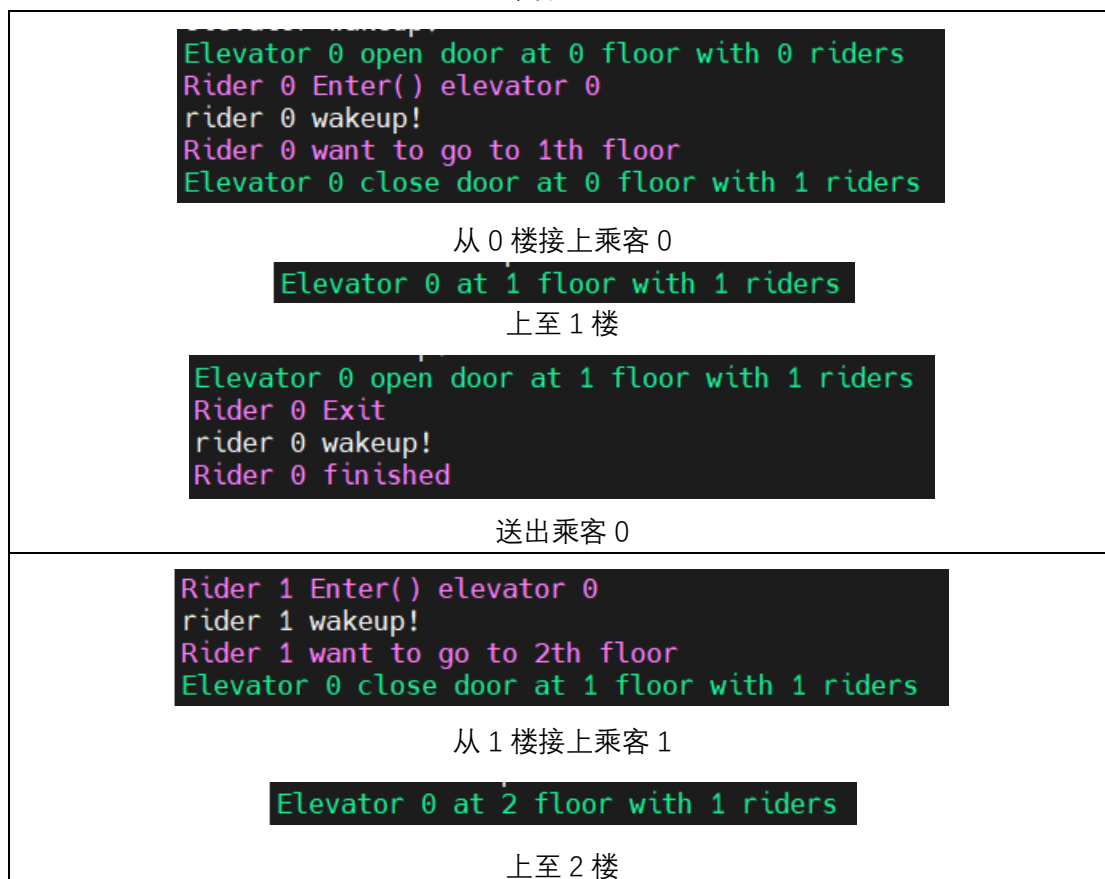
根据输出分析如下：

图表 16

乘客	起点	终点
0	0	1
1	1	2
2	2	1

电梯活动如下，观察以下流程可知是符合预期的：

图表 17



```
Elevator 0 open door at 2 floor with 1 riders
Rider 1 Exit
rider 1 wakeup!
Rider 1 finished
```

送出乘客 1

```
Rider 2 Enter() elevator 0
rider 2 wakeup!
Rider 2 want to go to 1th floor
Elevator 0 close door at 2 floor with 1 riders
```

从 2 楼接上乘客 2

```
Elevator 0 open door at 1 floor with 1 riders
```

下至 1 楼

```
Elevator 0 open door at 1 floor with 1 riders
Rider 2 Exit
rider 2 wakeup!
Rider 2 finished
```

送出乘客 2

3.4.3.4 两个电梯，容量 3，三个乘客，一共三层

首先改变为多个电梯模式：

```
code > threads > C Elevator.h > MULTIPLE_ELEVATOR
1  #ifndef ELEVATOR_H
2  #define ELEVATOR_H
3  #include "EventBarrier.h"
4  #include "system.h"
5  #include "synch.h"
6
7  // #define SINGLE_ELEVATOR
8  #define MULTIPLE_ELEVATOR // 单个或多个电梯
9  #define BOUNDED_ELEVATOR
10 // #define UNBOUNDED_ELEVATOR // 电梯容量是否有限
11 #define OPEN_AND_CLOSE_DOOR 20 // 电梯开关门的时间
12 #define RIDER_ENTER_OUT 10 // 一个乘客进出电梯的时间
13 #define ELEVATOR_MOVE_ONE_FLOOR 40 // 电梯移动一层楼的时间
14 #ifdef BOUNDED_ELEVATOR
15 #define ELEVATOR_CAPACITY 3 // 电梯容量
16 #endif
17 > enum ElevatorState...
23 void rider(int id, int srcFloor, int dstFloor);
24 > class Elevator...
61 > class Building...
97 #endif // ELEVATOR
```

图表 18

运行如下：


```
[ai22920212204180@ncore code]$ cd threads/
[ai22920212204180@ncore threads]$ ./naches -m 6 -f 3 -x 3 -b 2
Elevator 0 stop at 0 floor with 0 riders
Elevator 1 stop at 0 floor with 0 riders
rider 0 wakeup!

rider 0 start at 270 ticks from 0 floor to 1 floor
Rider 0 CallUp from 0th floor
Rider 0 wait to go up from 0th floor

Elevator 0 at 0 floor with 0 riders
rider 1 wakeup!

rider 1 start at 410 ticks from 1 floor to 2 floor
Rider 1 CallUp from 1th floor
Rider 1 wait to go up from 1th floor

rider 2 wakeup!
Elevator 1 at 0 floor with 0 riders

rider 2 start at 520 ticks from 2 floor to 1 floor
Rider 2 CallDown from 2th floor

Elevator 1 close door at 2 floor with 0 riders
elevator 0 wakeup!
elevator 1 wakeup!
Elevator 1 at 2 floor with 0 riders
Elevator 1 stop at 2 floor with 0 riders
elevator 0 wakeup!
Elevator 0 at 1 floor with 1 riders
elevator 0 wakeup!
Elevator 0 open door at 1 floor with 1 riders
Rider 2 Exit
rider 2 wakeup!
Rider 2 finished

rider 2 costs 31160

-----avg turnaround time 21893.33-----
-----max turnaround time 31160-----
-----min turnaround time 12620-----
Elevator 0 close door at 1 floor with 0 riders
```

图表 19

分析如下：

乘客情况如 3.4.3.3 中，电梯活动如下，符合预期：

图表 20

电梯 0 楼层	电梯 0 乘客	电梯 0 活动	电梯 1 楼层	电梯 1 乘客	电梯 1 活动
0	/	乘客 0 进入	0	/	/
0	0	/			
			1	/	乘客 1 进入
			1	1	/
1	0	乘客 0 出			
1	/	上楼			
			2	1	乘客 1 出
			2	/	/
2	/	乘客 2 进入			
2	2	下楼			
			2	/	停在 2 楼
1	2	乘客 2 出			
1	/	停在 1 楼			

3.4.3.5 问题回答

在此回答附加题中的问题：

若能被别的电梯顺路带走则不单独考虑；

若不能，则在所有 stop 状态电梯中选择距离最近的一个电梯。

4 实验总结

4.1 事件栅栏

事件栅栏是一种用于线程同步的工具，它在多线程环境中确保一组线程在某个关键点同步汇合。栅栏的特点是所有线程必须同时到达栅栏位置，然后才能继续执行后续任务。这种同步机制通过阻塞线程的方式，等待所有线程都准备好后一起释放，从而协调线程的执行。在多线程协同工作的场景中，事件栅栏可以用于确保所有线程在某个时刻同步执行，避免竞态条件和提高程序的可靠性。

4.2 闹钟原语

闹钟原语在多线程编程中用于实现线程的睡眠和唤醒操作。它允许线程在规定的时间内休眠，到达设定的唤醒时刻后，被自动唤醒。在 Nachos 系统中，Alarm 类通过模拟滴答来实现计时，提供了 Pause 函数，使线程能够在一段时间内休眠，实现了线程的同步。通过闹钟

原语，可以方便地控制线程的执行时间，避免了忙等待的浪费，提高了系统的效率。此外，为了防止系统终止，闹钟通常需要在单独的线程中工作，负责唤醒休眠的线程。

4.3 桥

汽车过桥问题涉及多辆车在一座狭窄的桥上单向行驶，需要设计同步方案以确保安全和避免竞态条件。在解决这一问题时，常采用先来先服务（FCFS）算法或者更复杂的红绿灯算法。FCFS 算法简单直观，但可能导致效率不高。红绿灯算法则通过模拟红绿灯的方式，动态调整通过桥的方向，更有效地利用资源。在实现过程中，需要考虑车辆的同向优化、桥上车辆数量限制、以及红绿灯切换的时机。整体设计旨在保障车辆的安全通行，提高系统的运行效率。

4.4 电梯

电梯问题涉及实现电梯控制器，包括单个电梯和支持多电梯的情况。在单电梯问题中，需要处理电梯停靠、乘客上下车、乘客等待电梯等多个同步关系。解决方案应避免乘客饥饿和竞态条件，确保电梯在停靠时等待所有离开的乘客并上车的乘客。在扩展到多电梯的情况下，需考虑电梯调度问题，即哪个电梯负责响应请求。通过使用同步机制，如锁、信号量、条件变量和事件栅栏，可以实现电梯问题的模拟。整体设计目标是模拟真实电梯系统，确保安全高效的乘客运输。

5 实验心得

通过实验，我深入理解了操作系统中的同步问题和线程调度。

- 实验一中，事件栅栏的应用使得线程能够同步到达特定点，这对于一些需要多线程协同工作的场景非常有用。
- 实验二中，闹钟的设计允许线程在指定的时间后自动唤醒，这在模拟时间敏感的操作时非常实用，如定时任务。
- 实验三中，通过模拟汽车过桥问题，我深刻理解了同步算法对系统效率的影响，尤其是红绿灯算法的应用，以及如何确保公平性和避免饥饿现象。
- 电梯问题是一个复杂的多线程协同问题，通过实现电梯控制器，我学会了如何处理多线程间的同步关系，包括线程等待、唤醒和电梯的合理调度。
- 通过对输出颜色和命令行参数的处理，我提升了对 C++ 语言的运用水平，使得程序更具可读性和交互性。

这些实验提供了丰富的经验，加深了我对操作系统中并发和同步机制的理解。

6 附录

6.1 threadtest.cc

```
// threadtest.cc
// Simple test case for the threads assignment.
//
// Create two threads, and have them context switch
// back and forth between themselves by calling Thread::Yield,
// to illustrate the inner workings of the thread system.
//
// Copyright (c) 1992-1993 The Regents of the University of California.
// All rights reserved. See copyright.h for copyright notice and
// limitation
// of liability and disclaimer of warranty provisions.
```

```

#include "copyright.h"
#include "system.h"
#include "dllist.h"
#include "Table.h"
/*
table.h 中包含表信息
*/
#include "BoundedBuffer.h"
#include "EventBarrier.h"
#include "Alarm.h"
#include "Bridge.h"
#include "Elevator.h"
// #define LAB_TWO
#define LAB_THREE
DLList *list;
BoundedBuffer *boundedbuffer;
EventBarrier *barrier;
extern void InsertList(DLList *list, int N);
extern void RemoveList(DLList *list, int N);
int nodenum;
int thread;
int testnum;
int error_type;
//-----
-
// SimpleThread
// Loop 5 times, yielding the CPU to another ready thread
// each iteration.
//
// "which" is simply a number identifying the thread, for debugging
// purposes.
//-----
-

void SimpleThread(int which)
{
    int num;

    for (num = 0; num < 5; num++)
    {
        printf("*** thread %d looped %d times\n", which, num);
        currentThread->Yield();
    }
}

```

```

// test are threads all empty
void TestDllist(int which)
{
    InsertList(list, nodenum);
    RemoveList(list, nodenum);
    if (!list->IsEmpty())
        printf("The thread %d :it has been empty ===\n\n", which);
    else
        printf("The thread %d :it hasn't been empty ===\n\n", which);
}

//-----
-
// ThreadTest1
// Set up a ping-pong between two threads, by forking a thread
// to call SimpleThread, and then calling SimpleThread ourselves.
//-----
-

void ThreadTest1()
{
    DEBUG('t', "Entering ThreadTest1");

    Thread *t = new Thread("forked thread");

    t->Fork(SimpleThread, 1);
    SimpleThread(0);
}

//-----
-
// ThreadTest2
//-----
-

void ThreadTest2()
{
    DEBUG('t', "Entering Multi-thread-test ");
    list = new DLList();
    for (int i = 0; i < thread; i++)
    {
        char No[4] = "1";
        // create the info
    }
}

```

```

        sprintf(No, "%d", i);
        char *name = new char[25];
        name[0] = '\0';
        strcat(name, "Thread ");
        strcat(name, No);
        Thread *t = new Thread(name);
        t->Fork(TestDllist, i);
    }
}

#ifdef LAB_TWO

/*
test3 测试 table
*/
void ThreadTest3(){
    DEBUG('t', "Entering Table-test");
    Table*t =new Table(3);//初始化
    for(int i=0;i<3;i++){//分配三个表项
        char No[4] = "1";
        // create the info
        sprintf(No, "%d", i);
        char *name = new char[25];
        name[0] = '\0';
        strcat(name, "Thread ");
        strcat(name, No);
        Thread *test_thread=new Thread(name);
        int x=t->Alloc(test_thread);//x 是申请成功后的下标
        ASSERT(x==i);//验证是否申请成功
        printf("allocation %d ok!\n",i);
    }
    for(int i=0;i<3;i++){//查看三个表项
        void *item=t->Get(i);
        if(item){//查看成功
            printf("get :%p \n",item);
        }else{//查看失败
            printf("get failed!\n");
        }
    }
    for(int i=0;i<3;i++){
        t->Release(i);
    }
}
/*

```

测试缓冲区

```
*/

void Buffer_producer(int size)
{
    for(int i=0;i<size;i++){
        char *context = new char[size+1];
        for (int j = 0; i < size; ++i){
            int x=Random()%10;
            context[j] = 'a' + x;//随机初始化一个字母
        }
        context[size] = '\0';
        boundedbuffer->Write(context, size);//写入缓冲区
        printf("above all, thread: %s write %s into buffer\n", currentThread->getName(), context);
    }
}

void Buffer_consumer(int size)
{
    for(int i=0;i<size;i++){
        char *context = new char[size+1];
        context[size] = '\0';
        boundedbuffer->Read(context, size);//从缓冲区读
        printf("above all, thread: %s read %s from buffer\n", currentThread->getName(), context);
    }
}

void ThreadTest4(){//先写后读
    printf("Entering Test-Buffer-write-first-read-second\n");
    boundedbuffer = new BoundedBuffer(10);
    for (int i = 0; i < 2; i++)//先分别写入 1, 2 个元素
    {
        char No[4] = "1";
        sprintf(No, "%d", i);
        char *name = new char[25];
        name[0] = '\0';
        strcat(name, "producer thread ");
        strcat(name, No);
        Thread *t = new Thread(name);
        t->Fork(Buffer_producer, i+1);
    }
}
```

```

    for (int i = 0; i < 2; i++)//后分别读出 1, 2 个元素
    {
        char No[4] = "1";
        sprintf(No, "%d", i);
        char *name = new char[25];
        name[0] = '\0';
        strcat(name, "consumer thread ");
        strcat(name, No);
        Thread *t = new Thread(name);
        t->Fork(Buffer_consumer, i+1);
    }
}

void ThreadTest5(){//先读后写
    printf("Entering Test-Buffer-read-first-write-second\n");
    boundedbuffer = new BoundedBuffer(10);
    for (int i = 0; i < 2; i++)//分别读出 1, 2 个元素
    {
        char No[4] = "1";
        sprintf(No, "%d", i);
        char *name = new char[25];
        name[0] = '\0';
        strcat(name, "consumer thread ");
        strcat(name, No);
        Thread *t = new Thread(name);
        t->Fork(Buffer_consumer, i+1);
    }
    for (int i = 0; i < 2; i++)//分别写入 1, 2 个元素
    {
        char No[4] = "1";
        sprintf(No, "%d", i);
        char *name = new char[25];
        name[0] = '\0';
        strcat(name, "producer thread ");
        strcat(name, No);
        Thread *t = new Thread(name);
        t->Fork(Buffer_producer, i+1);
    }
}

void ThreadTest6(){//读写均衡
    DEBUG('t', "Entering Test-Buffer-read-write-balance\n");
    boundedbuffer = new BoundedBuffer(10);
    for(int i=0;i<4;i++){

```

```

        if(i%2){//奇数读出 2 个元素
            char No[4] = "1";
            sprintf(No, "%d", i);
            char *name = new char[25];
            name[0] = '\0';
            strcat(name, "consumer thread ");
            strcat(name, No);
            Thread *t = new Thread(name);
            t->Fork(Buffer_consumer, 2);
        }else{//偶数写入 2 个元素
            char No[4] = "1";
            sprintf(No, "%d", i);
            char *name = new char[25];
            name[0] = '\0';
            strcat(name, "producer thread ");
            strcat(name, No);
            Thread *t = new Thread(name);
            t->Fork(Buffer_producer, 2);
        }
    }
}

/*
两生产者情况
*/
void test7fun1(int x){
    char *name = new char[25];
    strcat(name, "t0");
    Thread *t = new Thread(name, x);//优先级高
    t->Fork(Buffer_producer, 1);
}

void test7fun2(int x){
    char *name = new char[25];
    strcat(name, "t1");
    Thread *t = new Thread(name, x);//优先级低
    t->Fork(Buffer_producer, 1);
}

void ThreadTest7(){//优先级
    printf("Entering Test-Priority1\n");
    boundedbuffer = new BoundedBuffer(10);
    test7fun1(0);
    test7fun2(1);
}

```



```

}

void ThreadTest8(){//优先级
    printf("Entering Test-Priority2\n");
    boundedbuffer = new BoundedBuffer(10);
    test7fun1(1);
    test7fun2(0);
}

void test7fun3(int x){
    char *name = new char[25];
    strcat(name, "prodeucer-0");
    Thread *t = new Thread(name, x);//优先级低
    t->Fork(Buffer_producer, 1);
}

void test7fun4(int x){
    char *name = new char[25];
    strcat(name, "consumer-0");
    Thread *t = new Thread(name, x);//优先级高
    t->Fork(Buffer_consumer, 2);
}

void ThreadTest9(){//优先级
    printf("Entering Test-Priority1\n");
    boundedbuffer = new BoundedBuffer(10);
    test7fun3(1);
    test7fun4(0);
}
/*
一生产者一消费者，但消费者优先级更高
*/
#endif // LAB_TWO

#ifdef LAB_THREE
void TestEventBarrier_runing(int which)
{
    printf("\033[1;31m%s complete\033[0m\n", currentThread->getName());
    printf("waiters:%d\n", barrier->Waiters());
    barrier->Wait();
    for (int i = 0; i < 1000; ++i)
    {
        for (int j = 0; j < 1000; ++j)
        {

```

```

        currentThread->Yield();
        // thread runing;
    }
}
barrier->Complete();
printf("\033[1;32m%s finished\033[0m\n", currentThread->getName());
printf("waiters:%d\n", barrier->Waiters());
}

void TestEventBarrier_signal(int which)
{
    printf("\033[1;32m%s signaled\033[0m\n", currentThread->getName());
    barrier->Signal();
}

void ThreadTest3()
{
    Thread *tthread[4];
    barrier = new EventBarrier();
    tthread[0] = new Thread("thread 0");
    tthread[1] = new Thread("thread 1");
    tthread[0]->Fork(TestEventBarrier_runing, 0);
    tthread[1]->Fork(TestEventBarrier_runing, 1);
    tthread[2] = new Thread("thread 2");
    tthread[2]->Fork(TestEventBarrier_signal, 2);
}

/*alarm*/

void TestAlarm(int time)
{
    printf("\033[1;34mset %s at %dth tick, timeslide is %d\033[0m\n",
currentThread->getName(), stats->totalTicks, time); // 蓝色, set
    Alarm::instance->Pause(time);
    printf("\033[1;32m%s wake up at %dth tick\033[0m\n\n",
currentThread->getName(), // 绿色,起床
stats->totalTicks);
}

void ThreadTest4()
{
    Thread *tthread[5];
    Alarm::new_instance();
    tthread[0] = new Thread("thread 0");
    tthread[1] = new Thread("thread 1");

```

```

    tthread[2] = new Thread("thread 2");
    tthread[3] = new Thread("thread 3");
    tthread[4] = new Thread("thread 4");
    for(int i=0;i<5;i++){
        tthread[i]->Fork(TestAlarm, i+1);
    }
}

/*测试 bridge*/
int Total_num;      // 总测试的车辆数
int direc_0_percent; // 0 方向占的比例, x/10
int random_come_time = 10; // 随机时间间隔
int sumtime = 0;
int End_num = 0; // 成功过桥的数目
int should_start_time = 0; // 当前进程应当开始的时间
void TestBridge(int which)
{
    int rand_time = Random() % random_come_time; // 随机时间间隔
    should_start_time += rand_time;
    Alarm::instance->Pause(should_start_time);
    int start_time = stats->totalTicks;
    int rand = Random() % 10;
    int direc;
    if (rand < direc_0_percent)
        direc = 0;
    else
        direc = 1;
    printf("+++%s start at %d ticks in %ddirection\n\n",
currentThread->getName(), start_time, direc);
    Bridge::instance->OneVehicle(direc);
    printf("---%s costs %d\n\n", currentThread->getName(),
stats->totalTicks - start_time);
    sumtime += stats->totalTicks - start_time;
    End_num++; // 成功过桥的数目
    if (End_num == Total_num)
    {
        printf("\n-----avg turnaround time %.2f-----\n\n",
(float)sumtime / Total_num);
#ifdef BASIC_TRAFFIC_LIGHT || (defined ADV_TRAFFIC_LIGHT)
        Bridge::instance->finished = true; // 停止交通灯管理
#endif
    }
}

void ThreadTest5()

```

```

{
    Bridge::new_instance();
    Alarm::new_instance();
    for (int i = 0; i < Total_num; ++i)
    {
        char No[4] = "1";
        sprintf(No, "%d", i);
        char *name = new char[25];
        name[0] = '\0';
        strcat(name, "car ");
        strcat(name, No);
        Thread *t = new Thread(name);
        t->Fork(TestBridge, i); // 参数 i 实际上没做任何事
    }
}

/*elecator*/

int num_floors;          // 楼层数
int total_riders;        // 生成的总乘客数
int E_random_come_time = 2; // 随机时间间隔
int elevator_num;        // 电梯个数
long long int E_sumtime = 0; // 所有乘客一共用了多久
int E_End_num = 0; // 当前载了几个乘客
int E_should_start_time = 0; // 当前进程应当开始的时间
int max_turnaroundtime = 0;
int min_turnaroundtime = 2147483645;
void TestElevatorRider(int which)
{
    int srcfloor; // 用户起点
    int dstfloor; // 用户终点
    do
    {
        srcfloor = Random() % num_floors;
        dstfloor = Random() % num_floors;
    } while (srcfloor == dstfloor);
    int rand_time = Random() % E_random_come_time; // 随机时间间隔
    E_should_start_time += rand_time; // 随便一个开始时间
    Alarm::instance->Pause(E_should_start_time);
    int start_time = stats->totalTicks;
    printf("\033[1;35m\n%s start at %d ticks from %d floor to %d\n\033[0m", currentThread->getName(), start_time, srcfloor, dstfloor);
    rider(which, srcfloor, dstfloor);
}

```

```

        printf("\033[1;35m\n%s costs %d\n\033[0m",
currentThread->getName(),stats->totalTicks - start_time);
        int turnaroundtime = stats->totalTicks - start_time;
        if (turnaroundtime > max_turnaroundtime)
            max_turnaroundtime = turnaroundtime;
        if (turnaroundtime < min_turnaroundtime)
            min_turnaroundtime = turnaroundtime;
        E_sumtime += turnaroundtime;
        E_End_num++;
        if (E_End_num == total_riders)
        {
            printf("\n-----avg turnaround time %.2f-----\n",
                (float)E_sumtime / total_riders);
            printf("-----max turnaround time %d-----\n",
                max_turnaroundtime);
            printf("-----min turnaround time %d-----\n\n",
                min_turnaroundtime);
        }
    }
}

void TestElevatorControl(int which)
{
#ifdef MULTIPLE_ELEVATOR
    Building::instance->getElevator(which)->ElevatorControl();
#endif
#ifdef SINGLE_ELEVATOR
    Building::instance->getElevator()->ElevatorControl();
#endif
}

void ThreadTest6()
{
    Building::new_instance("building", num_floors, elevator_num);
    Alarm::new_instance();
#ifdef MULTIPLE_ELEVATOR
    for (int i = 0; i < elevator_num; i++)
    {
        char No[4] = "1";
        sprintf(No, "%d", i);
        char *name = new char[25];
        name[0] = '\0';
        strcat(name, "elevator ");
        strcat(name, No);
        Thread *t = new Thread(name);
        t->Fork(TestElevatorControl, i);
    }
}

```

```

#endif
#ifdef SINGLE_ELEVATOR
    Thread *t = new Thread("elevator");
    t->Fork(TestElevatorControl, 0);
#endif
    for (int i = 0; i < total_riders; ++i)
    {
        char No[4] = "1";
        sprintf(No, "%d", i);
        char *name = new char[25];
        name[0] = '\0';
        strcat(name, "rider ");
        strcat(name, No);
        Thread *t = new Thread(name);
        t->Fork(TestElevatorRider, i);
    }
}
#endif // LAB_THREE

void ThreadTest()
{
    switch (testnum)
    {
#ifdef LAB_TWO
        case 1:
            ThreadTest1();
            break;
        case 2:
            ThreadTest2();
            break;
        case 3:
            ThreadTest3();
            break;
        case 4:
            ThreadTest4();
            break;
        case 5:
            ThreadTest5();
            break;
        case 6:
            ThreadTest6();
            break;
        case 7:
            ThreadTest7();

```

```

        break;
    case 8:
        ThreadTest8();
        break;
    case 9:
        ThreadTest9();
        break;
#endif // LAB_TWO
#ifdef LAB_THREE
    case 3:
        ThreadTest3();
        break;
    case 4:
        ThreadTest4();
        break;
    case 5:
        ThreadTest5();
        break;
    case 6:
        ThreadTest6();
        break;
#endif // LAB_THREE
    default:
        printf("No test specified.\n");
        break;
}
}

```

代码 33

6.2 Elevator.h

```

#ifndef ELEVATOR_H
#define ELEVATOR_H
#include "EventBarrier.h"
#include "system.h"
#include "synch.h"

// #define SINGLE_ELEVATOR
#define MULTIPLE_ELEVATOR // 单个或多个电梯
#define BOUNDED_ELEVATOR
// #define UNBOUNDED_ELEVATOR // 电梯容量是否有限
#define OPEN_AND_CLOSE_DOOR 20 // 电梯开关门的时间
#define RIDER_ENTER_OUT 10 // 一个乘客进出电梯的时间
#define ELEVATOR_MOVE_ONE_FLOOR 40 // 电梯移动一层楼的时间
#ifdef BOUNDED_ELEVATOR

```

```

#define ELEVATOR_CAPACITY 3 // 电梯容量
#endif
enum ElevatorState
{
    UP,
    DOWN,
    STOP
};
void rider(int id, int srcFloor, int dstFloor);
class Elevator
{
public:
    friend class Building;
    Elevator(char *debugName, int numFloors, int myID);
    ~Elevator();
    char *getName() { return name; }
    int getId() { return ElevatorID; }
    // elevator control interface: called by Elevator thread
    void OpenDoors();           // signal exiters and enterers to action
    void CloseDoors();          // after exiters are out and enterers are
in
    void VisitFloor(int floor); // go to a particular floor
    // elevator rider interface (part 1): called by rider threads.
    bool Enter();               // get in
    void Exit();                // get out (iff destinationFloor)
    void RequestFloor(int floor); // tell the elevator our
destinationFloor
    // insert your methods here, if needed
    void ElevatorControl(); // 控制电梯运行
    int GetLastRequestFloor();

private:
    char *name;
    int currentfloor; // floor where currently stopped
    int occupancy;    // how many riders currently onboard
    // insert your data structures here, if needed
    int numFloors, ElevatorID;
    ElevatorState state;
    EventBarrier **ElevatorOutBarrier; // 出电梯的事件栅栏
    Lock *ElevatorLock;
    Condition *HaveRequest;
#ifdef MULTIPLE_ELEVATOR
    bool canEnter;
#endif
}

```



```

#ifdef BOUNDED_ELEVATOR
    Condition *ElevatorNotFull;
#endif
};
class Building
{
public:
    friend class Elevator;
    Building(char *debugname, int numFloors, int numElevators);
    ~Building();
    char *getName() { return name; }
    // elevator rider interface (part 2): called by rider threads
    void CallUp(int fromFloor);          // signal an elevator we want to
go up
    void CallDown(int fromFloor);        // ... down
    Elevator *AwaitUp(int fromFloor);    // wait for elevator arrival &
going up
    Elevator *AwaitDown(int fromFloor); // ... down
#ifdef SINGLE_ELEVATOR
    Elevator *getElevator();
#endif
#ifdef MULTIPLE_ELEVATOR
    Elevator *getElevator(int id);
#endif
    static void new_instance(char *debugname, int numFloors, int
numElevators);
    static Building *instance;

private:
    char *name;
#ifdef SINGLE_ELEVATOR
    Elevator *elevator; // the elevators in the building (array)
#endif
#ifdef MULTIPLE_ELEVATOR
    Elevator **elevator;
#endif
    EventBarrier **ElevatorUpBarrier; // 上行栅栏
    EventBarrier **ElevatorDownBarrier; // 下行栅栏
    // insert your data structures here, if needed
    int numFloors;
    int numElevators;
    Lock *BuildingLock;
};
#endif // ELEVATOR

```

6.3 Elevator.cc

```

#include "Elevator.h"
#include "Alarm.h"
Building *Building::instance;
#ifdef SINGLE_ELEVATOR
Elevator *Building::getElevator()
{
    return elevator;
}
void Building::new_instance(char *debugname, int numFloors, int
numElevators)
{
    instance = new Building(debugname, numFloors, numElevators);
}
Building::Building(char *debugname, int numFloors, int numElevators)
{
    name = debugname;
    this->numFloors = numFloors;
    this->numElevators = numElevators;
    elevator = new Elevator("elevator", numFloors, 0);// 因为只有一个电
梯, 所以 id 是 0
    BuildingLock = new Lock("Building lock");
    ElevatorUpBarrier = new EventBarrier *[numFloors];// 每层楼两个栅栏
    ElevatorDownBarrier = new EventBarrier *[numFloors];
    for (int i = 0; i < numFloors; ++i)
    {
        ElevatorUpBarrier[i] = new EventBarrier;
        ElevatorDownBarrier[i] = new EventBarrier;
    }
}
Building::~Building()
{
    delete elevator;
    delete BuildingLock;
    for (int i = 0; i < numFloors; ++i)
    {
        delete ElevatorUpBarrier[i];
        delete ElevatorDownBarrier[i];
    }
    delete[] ElevatorUpBarrier;
    delete[] ElevatorDownBarrier;
}

```

```

Elevator *Building::AwaitDown(int fromFloor)
{
    ElevatorDownBarrier[fromFloor]->Wait();// 休眠，并伺机而动
    return elevator;
}
Elevator *Building::AwaitUp(int fromFloor)
{
    ElevatorUpBarrier[fromFloor]->Wait();
    return elevator;
}
void Building::CallDown(int fromFloor)
{
    elevator->ElevatorLock->Acquire();
    elevator->HaveRequest->Signal(elevator->ElevatorLock);
    elevator->ElevatorLock->Release();
}
void Building::CallUp(int fromFloor)
{
    elevator->ElevatorLock->Acquire();
    elevator->HaveRequest->Signal(elevator->ElevatorLock);
    elevator->ElevatorLock->Release();
}
#endif // SINGLE_ELEVATOR
#ifdef MULTIPLE_ELEVATOR
Elevator *
Building::getElevator(int id)
{
    return elevator[id];
}
void Building::new_instance(char *debugname, int numFloors, int
numElevators)
{
    instance = new Building(debugname, numFloors, numElevators);
}
Building::Building(char *debugname, int numFloors, int numElevators)
{
    name = debugname;
    this->numFloors = numFloors;
    this->numElevators = numElevators;
    ElevatorUpBarrier = new EventBarrier *[numFloors];
    ElevatorDownBarrier = new EventBarrier *[numFloors];
    elevator = new Elevator *[numElevators];
    for (int i = 0; i < numElevators; ++i)
    {

```

```

        elevator[i] = new Elevator("elevator", numFloors, i);
    }
    for (int i = 0; i < numFloors; ++i)
    {
        ElevatorUpBarrier[i] = new EventBarrier;
        ElevatorDownBarrier[i] = new EventBarrier;
    }
    BuildingLock = new Lock("Building lock");
}
Building::~~Building()
{
    for (int i = 0; i < numElevators; ++i)
    {
        delete elevator[i];
    }
    for (int i = 0; i < numFloors; ++i)
    {
        delete ElevatorUpBarrier[i];
        delete ElevatorDownBarrier[i];
    }
    delete[] elevator;
    delete[] ElevatorUpBarrier;
    delete[] ElevatorDownBarrier;
    delete BuildingLock;
}
Elevator *Building::AwaitDown(int fromFloor)
{
    ElevatorDownBarrier[fromFloor]->Wait();
    BuildingLock->Acquire();
    Elevator *re;
    for (int i = 0; i < numElevators; ++i)
    {
        if (elevator[i]->currentfloor == fromFloor &&
(elevator[i]->state == DOWN || elevator[i]->state == STOP) &&
elevator[i]->canEnter)
        {
            re = elevator[i];
            break;
        }
    }
    BuildingLock->Release();
    return re;
}
Elevator *Building::AwaitUp(int fromFloor)

```

```

{
    ElevatorUpBarrier[fromFloor]->Wait();
    BuildingLock->Acquire();
    Elevator *re;
    for (int i = 0; i < numElevators; ++i)
    {
        if (elevator[i]->currentfloor == fromFloor &&
            (elevator[i]->state == UP || elevator[i]->state == STOP) &&
            elevator[i]->canEnter)
        {
            re = elevator[i];
            break;
        }
    }
    BuildingLock->Release();
    return re;
}

void Building::CallDown(int fromFloor)
{
    BuildingLock->Acquire();
    int MinInterval = numFloors + 1;
    int eleNum = -1;
    for (int i = 0; i < numElevators; ++i)
    {
        if (elevator[i]->state == STOP &&
            abs(elevator[i]->currentfloor - fromFloor) < MinInterval)
        {
            MinInterval = abs(elevator[i]->currentfloor - fromFloor);
            eleNum = i;
        }
    }
    if (eleNum != -1)
    {
        elevator[eleNum]->HaveRequest->Signal(BuildingLock);
    }
    BuildingLock->Release();
}

void Building::CallUp(int fromFloor)
{
    BuildingLock->Acquire();
    int MinInterval = numFloors + 1;
    int eleNum = -1;
    for (int i = 0; i < numElevators; ++i)
    {

```

```

        if (elevator[i]->state == STOP &&
            abs(elevator[i]->currentfloor - fromFloor) < MinInterval)
        {
            MinInterval = abs(elevator[i]->currentfloor - fromFloor);
            eleNum = i;
        }
    }
    if (eleNum != -1)
    {
        elevator[eleNum]->HaveRequest->Signal(BuildingLock);
    }
    BuildingLock->Release();
}
#endif // MULTIPLE_ELEVATOR
Elevator::Elevator(char *debugName, int numFloors, int myID)
{
    name = debugName;
    this->numFloors = numFloors;
    ElevatorID = myID;
    ElevatorOutBarrier = new EventBarrier *[numFloors];
    for (int i = 0; i < numFloors; ++i)
    {
        ElevatorOutBarrier[i] = new EventBarrier;
    }
    ElevatorLock = new Lock("Elevator lock");
    state = STOP;
    currentfloor = 0;
    HaveRequest = new Condition("have a elevator request");
    occupancy = 0;
#ifdef MULTIPLE_ELEVATOR
    canEnter = false;
#endif
#ifdef BOUNDED_ELEVATOR
    ElevatorNotFull = new Condition("elevator is not full");
#endif
}
Elevator::~~Elevator()
{
    for (int i = 0; i < numFloors; ++i)
    {
        delete ElevatorOutBarrier[i];
    }
    delete[] ElevatorOutBarrier;
    delete[] ElevatorLock;
}

```

```

        delete HaveRequest;
#ifdef BOUNDED_ELEVATOR
        delete ElevatorNotFull;
#endif
    }
    void Elevator::OpenDoors()
    {
        Alarm::instance->Pause(OPEN_AND_CLOSE_DOOR);
        printf("\033[1;32mElevator %d open door at %d floor with %d
riders\n\033[0m", ElevatorID, currentfloor, occupancy);
        if (ElevatorOutBarrier[currentfloor]->Waiters() != 0)
        {
            ElevatorOutBarrier[currentfloor]->Signal();
        }
#ifdef MULTIPLE_ELEVATOR
        canEnter = true;
#endif
        if (state == UP)
        {
            if
(Building::instance->ElevatorUpBarrier[currentfloor]->Waiters() != 0)
            {
                Building::instance->ElevatorUpBarrier[currentfloor]->Signal(
);
            }
        }
        else if (state == DOWN)
        {
            if
(Building::instance->ElevatorDownBarrier[currentfloor]->Waiters() != 0)
            {
                Building::instance->ElevatorDownBarrier[currentfloor]->Signa
l();
            }
        }
    }
    void Elevator::CloseDoors()
    {
#ifdef MULTIPLE_ELEVATOR
        canEnter = false;
#endif
        printf("\033[1;32mElevator %d close door at %d floor with %d
riders\033[0m\n\n", ElevatorID, currentfloor, occupancy);
        Alarm::instance->Pause(OPEN_AND_CLOSE_DOOR);

```

```

}
void Elevator::VisitFloor(int floor)
{
#ifdef BOUNDED_ELEVATOR
    ElevatorLock->Acquire();
    ElevatorNotFull->Broadcast(ElevatorLock);
    ElevatorLock->Release();
#endif
    Alarm::instance->Pause(ELEVATOR_MOVE_ONE_FLOOR);
    currentfloor = floor;
}
bool Elevator::Enter()
{
    occupancy++;
#ifdef BOUNDED_ELEVATOR
    if (occupancy > ELEVATOR_CAPACITY)
    {
        occupancy--;
        if (state == UP) // 保证 complete()的数目和 Wait 相同
        {
            Building::instance->ElevatorUpBarrier[currentfloor]->Complete();
        }
        else if (state == DOWN)
        {
            Building::instance->ElevatorDownBarrier[currentfloor]->Complete();
        }
        ElevatorLock->Acquire();
        ElevatorNotFull->Wait(ElevatorLock); // 阻塞超载
        ElevatorLock->Release();
        return false; // 用于换乘
    }
#endif
    Alarm::instance->Pause(RIDER_ENTER_OUT);
    if (state == UP)
    {
        Building::instance->ElevatorUpBarrier[currentfloor]->Complete();
    }
    else if (state == DOWN)
    {
        Building::instance->ElevatorDownBarrier[currentfloor]->Complete();
    }
}

```



```

        return true;
    }
    void Elevator::Exit()
    {
        Alarm::instance->Pause(RIDER_ENTER_OUT);
        occupancy--;
        ElevatorOutBarrier[currentfloor]->Complete();
    }
    void Elevator::RequestFloor(int floor)
    {
        ElevatorOutBarrier[floor]->Wait();
    }
    int Elevator::GetLastRequestFloor()// 确定目标
    {
        ElevatorLock->Acquire();
        int return_value = -1;// 最开始是-1
        if (state == STOP)
        {
            for (int i = 0; i < numFloors; ++i)
            {
                if (Building::instance->ElevatorUpBarrier[i]->Waiters() != 0
                || ElevatorOutBarrier[i]->Waiters() != 0 ||
                    Building::instance->ElevatorDownBarrier[i]->Waiters() !=
0)
                {
                    return_value = i;
                    break;
                }
            }
        }
        else if (state == UP)
        {
            for (int i = numFloors - 1; i >= currentfloor; --i)
            {
                if (Building::instance->ElevatorDownBarrier[i]->Waiters() !=
0 ||
                    ElevatorOutBarrier[i]->Waiters() != 0 ||
Building::instance->ElevatorUpBarrier[i]->Waiters() != 0)
                {
                    return_value = i;
                    break;
                }
            }
        }
    }
}

```

```

else
{
    for (int i = 0; i <= currentfloor; ++i)
    {
        if (Building::instance->ElevatorDownBarrier[i]->Waiters() !=
            0 ||
            ElevatorOutBarrier[i]->Waiters() != 0 ||
Building::instance->ElevatorUpBarrier[i]->Waiters() != 0)
        {
            return_value = i;
            break;
        }
    }
}
ElevatorLock->Release();
return return_value;
}

void Elevator::ElevatorControl()// 最核心函数
{
    bool no_requeset_flag = false;
    while (true)
    {
        int dest_floor = GetLastRequestFloor();
        if (state == STOP)
        {
            ElevatorLock->Acquire();
            printf("\033[1;32mElevator %d stop at %d floor with %d
riders\n\033[0m", ElevatorID, currentfloor, occupancy);
            if (dest_floor == -1)
            {
                HaveRequest->Wait(ElevatorLock);// 等人来
            }
            ElevatorLock->Release();
            printf("\033[1;32mElevator %d at %d floor with %d
riders\n\033[0m", ElevatorID, currentfloor, occupancy);
            dest_floor = GetLastRequestFloor();// 等待电梯目标, 没有目标的
话就会奔着-1层去, -1的情况会被特殊处理
            if (dest_floor > currentfloor)
                state = UP;
            else
                state = DOWN;
        }
        else if (state == UP)
        {

```

```

        while (currentfloor != dest_floor && dest_floor != -1)
        {
            no_requset_flag = false;
            if
(Building::instance->ElevatorUpBarrier[currentfloor]->Waiters() != 0
|| ElevatorOutBarrier[currentfloor]->Waiters() != 0)
            { // 我是上升的电梯，你是上升的乘客或者要出电梯的乘客
              // 根据常识，此时遇到下行的乘客，电梯不予理睬
                OpenDoors();
                CloseDoors();
            }
            VisitFloor(currentfloor + 1);
            dest_floor = GetLastRequestFloor();
            printf("\033[1;32mElevator %d at %d floor with %d
riders\n\033[0m",
                ElevatorID, currentfloor, occupancy);
        }
        if (dest_floor == -1) // 有多个电梯时，若请求被其他电梯满足，则
        会被取消，故需在此处判断
        {
            if (no_requset_flag) // 若在上一个循环中判断的另外一个方向也
            无请求
            {
                state = STOP;
                continue;
            }
            state = DOWN; // 不知道去哪，换个方向吧
            no_requset_flag = true;
            continue;
        }
        if
(Building::instance->ElevatorUpBarrier[currentfloor]->Waiters() == 0)
        { // 到达目的地了，没乘客就调头，看另一个方向有无乘客
            state = DOWN; // 调头
            OpenDoors();
            CloseDoors();
            dest_floor = GetLastRequestFloor();
            if (dest_floor == -1) // 若 DOWN 方向上也无新的请求则使电梯
            停下
            {
                state = STOP;
            }
            else // 否则向 DOWN 方向运行
                VisitFloor(currentfloor - 1);
        }
        else // 有想从当前层向上的

```

```

        {
            OpenDoors();
            CloseDoors();
            VisitFloor(currentfloor + 1);
        }
        printf("\033[1;32mElevator %d at %d floor with %d
riders\n\033[0m",
            ElevatorID, currentfloor, occupancy);
    }
    else
    {
        while (currentfloor != dest_floor && dest_floor != -1)
        {
            no_requset_flag = false;
            if
(Building::instance->ElevatorDownBarrier[currentfloor]->Waiters() != 0
|| ElevatorOutBarrier[currentfloor]->Waiters() !=
0)
            {
                OpenDoors();
                CloseDoors();
            }
            VisitFloor(currentfloor - 1);
            dest_floor = GetLastRequestFloor();
            printf("\033[1;32mElevator %d at %d floor with %d
riders\n\033[0m", ElevatorID, currentfloor, occupancy);
        }
        if (dest_floor == -1)
        {
            if (no_requset_flag)
            {
                state = STOP;
                continue;
            }
            state = UP;
            no_requset_flag = true;
            continue;
        }
        if
(Building::instance->ElevatorDownBarrier[currentfloor]->Waiters() == 0)
        {
            state = UP;
            OpenDoors();

```

```

        CloseDoors();
        dest_floor = GetLastRequestFloor();
        if (dest_floor == -1)
            state = STOP;
        else
            VisitFloor(currentfloor + 1);
    }
    else
    {
        OpenDoors();
        CloseDoors();
        VisitFloor(currentfloor - 1);
    }
    printf("\033[1;32mElevator %d at %d floor with %d
riders\n\033[0m", ElevatorID, currentfloor, occupancy);
}
currentThread->Yield();
}
}

void rider(int id, int srcFloor, int dstFloor)
{
    Elevator *e;
    // printf("src:%d dst:%d\n",srcFloor,dstFloor);
    do
    {
        if (srcFloor < dstFloor)
        {
            printf("\033[1;35mRider %d CallUp from %dth floor\n\033[0m",
id, srcFloor);
            Building::instance->CallUp(srcFloor); // 呼叫电梯
            printf("\033[1;35mRider %d wait to go up from %dth
floor\n\n\033[0m", id, srcFloor);
            e = Building::instance->AwaitUp(srcFloor); // 电梯来了
        }
        else
        {
            printf("\033[1;35mRider %d CallDown from %dth
floor\n\n\033[0m", id, srcFloor);
            Building::instance->CallDown(srcFloor);
            printf("\033[1;35mRider %d want to go down to %dth
flooe\033[0m\n\n", id, srcFloor);
            e = Building::instance->AwaitDown(srcFloor);
        }
    }
}

```

```

        printf("\033[1;35mRider %d Enter() elevator %d\n\033[0m", id,
e->getId());
    } while (!e->Enter()); // elevator might be full!
    printf("\033[1;35mRider %d want to go to %dth floor\n\033[0m", id,
dstFloor);
    e->RequestFloor(dstFloor);
    printf("\033[1;35mRider %d Exit\n\033[0m", id);
    e->Exit();
    printf("\033[1;35mRider %d finished\n\033[0m", id);
}

```

代码 35

6.4 EventBarrier.h

```

#ifndef EventBarrier_H
#define EventBarrier_H
#include "synch.h"
#include "system.h"
class EventBarrier
{
public:
    EventBarrier();
    ~EventBarrier();
    void Wait(); // Wait until the event is signaled.
                // Return immediately if already in the signaled state.
    void Signal(); // Signal the event and block until all threads
                // that wait for this event have responded.
                // The EventBarrier reverts to the unsignaled state when Signal()
returns.
    void Complete(); // indicate that the calling thread
                // has finished responding to a signaled event, and block
                // until all other threads that wait for this event have also
responded.
    int Waiters(); // Return a count of threads that are waiting
                // for the event or that have not yet responded to it.
private:
    int waiternum;
    Lock *lock; // 一个队列
    Condition *event, *comp; // two conditions
    bool signaled;
};

#endif

```

代码 36

6.5 EventBarrier.cc

```
#include "EventBarrier.h"
EventBarrier::EventBarrier()
{
    waiternum = 0;
    lock = new Lock("EventBarrier Lock");
    event = new Condition("Event come cond");
    comp = new Condition("Event complete cond");
    signaled = false;
}
EventBarrier::~EventBarrier()
{
    delete lock;
    delete event;
    delete comp;
}
void EventBarrier::Wait()
{
    lock->Acquire();
    waiternum++;
    if (signaled) // 如果事件栅栏已经处于 Signaled 状态，直接返回
    {
        lock->Release();
        return;
    }
    event->Wait(lock);
    lock->Release();
}
void EventBarrier::Signal()
{
    lock->Acquire();
    signaled = true;
    event->Broadcast(lock);
    comp->Wait(lock);
    signaled = false;
    lock->Release();
}
void EventBarrier::Complete()
{
    lock->Acquire();
    waiternum--;
    if (waiternum == 0)
    {
        comp->Broadcast(lock); // 唤醒所有线程
    }
}
```

```

    }
    else
    {
        comp->Wait(lock); // 当前线程休眠
    }
    lock->Release();
}
int EventBarrier::Waiters()
{
    return waiternum;
}

```

代码 37

6.6 Alarm.h

```

#ifndef ALARM_H
#define ALARM_H
#include "system.h"
#include "list.h"

void check(int which);
class Alarm
{
public:
    Alarm();
    ~Alarm();
    void Pause(int howLong);
    List *queue;
    // Timer *timer;
    int waiters;
    void Wakeup();
    static void new_instance();
    static Alarm *instance;
    static void Continue(int arg);
};
#endif

```

代码 38

6.7 Alarm.cc

```

#include "Alarm.h"
#include "interrupt.h"

Alarm *Alarm::instance;

void Alarm::new_instance()
{

```



```

    instance = new Alarm();
}

Alarm::Alarm()
{
    queue = new List;
    waiters = 0;
}

Alarm::~~Alarm()
{
    delete queue;
}

void Alarm::Continue(int arg){// interrupt 要用得的 handler
    Alarm* temp = (Alarm*)arg;
    temp->Wakeup();
}

void Alarm::Pause(int howLong) // howlong 单位为中断次数
{
    waiters++;
    Thread *loop_t;
    if (waiters == 1)
    {
        loop_t = new Thread("loop thread");
        loop_t->Fork(check, 0); // 使至少有一个线程在运行，防止系统终止
    }
    IntStatus oldLevel = interrupt->SetLevel(IntOff); // 必须关中断
    int Ticks = TimerTicks * howLong; // timerticks 默认 100
    queue->SortedInsert((void *)currentThread,
                        stats->totalTicks + Ticks); // 按结束时间依序插入
链表
    interrupt->Schedule(Continue, (int)this, Ticks, TimerInt); // 用这一
行在 howlong 以后唤醒 sleep 的进程
    currentThread->Sleep();
    (void)interrupt->SetLevel(oldLevel);
}

void check(int which)
{
    while (Alarm::instance->waiters != 0)
    {
        currentThread->Yield();
    }
}

```

```

    }
    currentThread->Finish();
}

void Alarm::Wakeup()
{
    int duetime = -1;
    Thread *thread = NULL;
    IntStatus oldLevel = interrupt->SetLevel(IntOff); // 必须关中断
    thread = (Thread *)queue->SortedRemove(&duetime);
    // 把 duetime 改成被删除的那个元素的 key 值，该 key 值即插入时间，将该值作为
thread 的名字
    waiters--;
    scheduler->ReadyToRun(thread);
    printf("%s wakeup!\n", thread->getName());
    (void)interrupt->SetLevel(oldLevel);
}

```

代码 39

6.8 Bridge.h

```

#ifndef BRIDGE_H
#define BRIDGE_H
#include "system.h"
#include "synch.h"
#include "Alarm.h"
#include "../machine/stats.h"
#define CROSS_BRIDGE_TIME 10
#define BASE_GREEN_LIGHT_TIME (10 * CROSS_BRIDGE_TIME) // 基准绿灯时间,为
服务时间的十倍
// #define FCFS //先来先服务算法
// #define ONE_DIRECTION //同方向优化算法
// #define BASIC_TRAFFIC_LIGHT //基础版红绿灯算法
#define ADV_TRAFFIC_LIGHT // 改进版红绿灯算法（动态调节）
#if (defined BASIC_TRAFFIC_LIGHT) || (defined ADV_TRAFFIC_LIGHT)
void TrafficLightManager(int which);
#endif
class Bridge
{
public:
    Bridge();
    ~Bridge();
    void OneVehicle(int direc)
    {
        ArriveBridge(direc);
    }

```

```

        CrossBridge(direc);
        ExitBridge(direc);
    };
    static void new_instance();// 方便通过类名访问
    static Bridge *instance;

private:
    void ArriveBridge(int direc);
    void CrossBridge(int direc);
    void ExitBridge(int direc);
#ifdef FCFS // 先来先服务算法
    int on_bridge_num;
    int current_direc;
    bool pre_is_wait;
    Lock *lock;
    Condition *con;
#endif
#ifdef ONE_DIRECTION // 同方向优化算法
    int on_bridge_num;
    int current_direc;
    Lock *lock;
    Condition *con;
#endif
#ifdef BASIC_TRAFFIC_LIGHT // 基础版红绿灯算法
public:
    int on_bridge_num;
    int current_direc;
    int next_switch_time;// 下次切换时间
    bool yellow_light_on;
    Lock *lock;
    Condition *direc_con[2];//
    int direc_car_num[2];
    void switch_status();
    void CheckIfDue();
    bool finished;
    Thread *manager; // 红绿灯管理进程
#endif
#ifdef ADV_TRAFFIC_LIGHT
public:
    Thread *manager;
    int on_bridge_num;
    int current_direc;
    int next_switch_time;
    bool yellow_light_on;

```

```

    Lock *lock;
    Condition *direc_con[2];
    int direc_car_num[2];    // 用来记录某个方向有多少车
    int direc_car_can_go[2]; // 用来记录还剩多少可以放行的车
    void switch_status();
    void CheckIfDue();
    int switchCount; // 循环次数
    bool finished;
#endif
};

#endif // BRIDGE_H

```

代码 40

6.9 Bridge.cc

```

#include "Bridge.h"

Bridge *Bridge::instance;
void Bridge::new_instance()
{
    instance = new Bridge();
}
#ifdef FCFS // 先来先服务算法
Bridge::Bridge()
{
    lock = new Lock("bridge lock");
    con = new Condition("FCFS condition");
    on_bridge_num = 0;
    current_direc = 0;
    pre_is_wait = false;
}
Bridge::~Bridge()
{
    delete lock;
    delete con;
}
void Bridge::ArriveBridge(int direc)
{
    lock->Acquire();
    printf("\033[1;34m%s car arrived the bridge in %d direc\033[0m\n\n",
        currentThread->getName(), direc);
    if (on_bridge_num == 0)
    {
        current_direc = direc;
    }
}

```

```

    }
    while (direc != current_direc || on_bridge_num >= 3 || pre_is_wait)
    {
        pre_is_wait = true; // 下一辆车的前面有车
        con->Wait(lock); // 等待合适的时机再开始工作
        pre_is_wait = false; // 轮到我了!
        if (on_bridge_num == 0)
        {
            current_direc = direc;
        }
    }
    on_bridge_num++;
    lock->Release();
}

void Bridge::CrossBridge(int direc)
{
    printf("\033[1;32m%s car is crossing the bridge in %d\n",
        direc, currentThread->getName(), direc);
    Alarm::instance->Pause(CROSS_BRIDGE_TIME); // 在桥上走 10s
}

void Bridge::ExitBridge(int direc)
{
    lock->Acquire();
    on_bridge_num--;
    printf("\033[1;33m%s car leave the bridge\n",
        currentThread->getName());
    con->Broadcast(lock); // 把被阻塞的线程一一取出
    lock->Release();
}

#endif

#ifdef ONE_DIRECTION // 同方向优化算法
Bridge::Bridge()
{
    lock = new Lock("bridge lock");
    con = new Condition("ONE_DIRECTION condition");
    on_bridge_num = 0;
    current_direc = 0;
}

Bridge::~Bridge()
{
    delete lock;
    delete con;
}

```

```

void Bridge::ArriveBridge(int direc)
{
    lock->Acquire();
    printf("\033[1;34m%s car arrived the bridge in %d direc\033[0m\n\n",
        currentThread->getName(), direc);
    if (on_bridge_num == 0)
    {
        current_direc = direc;
    }
    while (direc != current_direc || on_bridge_num >= 3)
    {
        con->Wait(lock);
        if (on_bridge_num == 0)
        {
            current_direc = direc;
        }
    }
    on_bridge_num++;
    lock->Release();
}

void Bridge::CrossBridge(int direc)
{
    printf("\033[1;32m%s car is crossing the bridge in %d
direc\033[0m\n\n",
        currentThread->getName(), direc);
    Alarm::instance->Pause(CROSS_BRIDGE_TIME);
}

void Bridge::ExitBridge(int direc)
{
    lock->Acquire();
    on_bridge_num--;
    printf("\033[1;33m%s car leave the bridge\033[0m\n\n",
currentThread->getName());
    con->Broadcast(lock);
    lock->Release();
}

#endif

#ifdef BASIC_TRAFFIC_LIGHT // 基础版红绿灯算法
void TrafficLightManager(int which)// 打开交通灯
{
    while (!Bridge::instance->finished)
    {
        Bridge::instance->CheckIfDue();
        currentThread->Yield();
    }
}

```

```

    }
}
Bridge::Bridge()
{
    lock = new Lock("bridge lock");
    direc_con[0] = new Condition("direc 0 condition");
    direc_con[1] = new Condition("direc 1 condition");
    on_bridge_num = 0;
    current_direc = 1; // 默认 0 方向先行,由于最开始要切换一次方向,故先设成 1
    next_switch_time = 0;
    yellow_light_on = false;
    direc_car_num[0] = 0;
    direc_car_num[1] = 0;
    manager = new Thread("traffic light manager thread");
    manager->Fork(TrafficLightManager, 0); // 打开交通灯,直到所有车辆过桥
    finished = false;
}
Bridge::~Bridge()
{
    delete lock;
    delete direc_con[0];
    delete direc_con[1];
}
void Bridge::ArriveBridge(int direc)
{
    CheckIfDue();
    lock->Acquire();
    printf("\033[1;34m%s car arrived the bridge in %d direc\033[0m\n\n",
           currentThread->getName(), direc);
    while (direc != current_direc || on_bridge_num >= 3 ||
yellow_light_on)
    {
        direc_car_num[direc]++;
        direc_con[direc]->Wait(lock);
        direc_car_num[direc]--;
    }
    on_bridge_num++;
    lock->Release();
}
void Bridge::CrossBridge(int direc)
{
    printf("\033[1;32m%s car is crossing the bridge in %d
direc\033[0m\n\n",
           currentThread->getName(), direc);

```

```

        Alarm::instance->Pause(CROSS_BRIDGE_TIME);
    }
void Bridge::ExitBridge(int direc)// 离开
{
    lock->Acquire();
    on_bridge_num--;
    printf("\033[1;33m%s car leave the bridge\033[0m\n\n",
currentThread->getName());
    direc_con[direc]->Broadcast(lock);
    lock->Release();
    CheckIfDue();
}
void Bridge::CheckIfDue()
{
    lock->Acquire();
    if (stats->totalTicks >= next_switch_time)
    {
        yellow_light_on = true;
        if (on_bridge_num == 0)
        {
            yellow_light_on = false;
            switch_status();// 切换方向
        }
    }
    lock->Release();
}
void Bridge::switch_status()
{
    current_direc = (current_direc + 1) % 2; // 切换方向
    next_switch_time = stats->totalTicks + BASE_GREEN_LIGHT_TIME *
TimerTicks;
    printf("\033[1;31mnow switch the direc to %d\n\033[0m",
current_direc);
    printf("\033[1;31mnext switch at %d ticks\n\033[0m",
next_switch_time);
    printf("\033[1;31m0 wait cars %d\t1 wait cars %d\n\033[0m",
        direc_car_num[0], direc_car_num[1]);
    direc_con[current_direc]->Broadcast(lock);
}
#endif
#ifdef ADV_TRAFFIC_LIGHT // 改进版红绿灯算法（动态调节）
void TrafficLightManager(int whitch)
{
    while (!Bridge::instance->finished)

```



```

    {
        Bridge::instance->lock->Acquire();
        Bridge::instance->CheckIfDue();
        Bridge::instance->lock->Release();
        currentThread->Yield();
    }
}

Bridge::Bridge()
{
    lock = new Lock("bridge lock");
    direc_con[0] = new Condition("direc 0 condition");
    direc_con[1] = new Condition("direc 1 condition");
    on_bridge_num = 0;
    current_direc = 1; // 默认 0 方向先行,由于最开始要切换一次方向, 故先设成 1
    yellow_light_on = false;
    next_switch_time = 0;
    direc_car_num[0] = 0; // 在切换桥上红绿灯之前, 该方向能走的车辆
    direc_car_num[1] = 0;
    direc_car_can_go[0] = 0; // 切换红绿灯之后, 该方向能走的车辆, 只有减少
    direc_car_can_go[1] = 0;
    finished = false;
    switchCount = 0;
    manager = new Thread("traffic light manager thread");
    manager->Fork(TrafficLightManager, 0);
}

Bridge::~Bridge()
{
    delete lock;
    delete direc_con[0];
    delete direc_con[1];
}

void Bridge::ArriveBridge(int direc)
{
    lock->Acquire();
    printf("\033[1;34m%s car arrived the bridge in %d direc\033[0m\n\n",
        currentThread->getName(), direc);
    direc_car_num[direc]++;
    while (direc != current_direc || on_bridge_num >= 3 ||
yellow_light_on || (direc_car_can_go[direc] <= 0 && switchCount > 1))
    {
        CheckIfDue();
        direc_con[direc]->Wait(lock);
    }
    if (switchCount <= 1)

```

```

    {
        direc_car_num[direc]--;
    }
    direc_car_can_go[direc]--;
    on_bridge_num++;
    lock->Release();
}

void Bridge::CrossBridge(int direc)
{
    printf("\033[1;32m%s car is crossing the bridge in %d\n",
        direc,
        currentThread->getName(), direc);
    Alarm::instance->Pause(CROSS_BRIDGE_TIME);
}

void Bridge::ExitBridge(int direc)
{
    lock->Acquire();
    on_bridge_num--;
    // printf("---%s car leave the bridge at %d\n",
currentThread->getName(),
    //      stats->totalTicks);
    printf("\033[1;33m%s car leave the bridge\n",
currentThread->getName());
    direc_con[direc]->Broadcast(lock);
    CheckIfDue();
    lock->Release();
}

void Bridge::CheckIfDue()
{
    if (switchCount <= 1)
    {
        if (stats->totalTicks >= next_switch_time)
        {
            yellow_light_on = true;
            if (on_bridge_num == 0)
            {
                yellow_light_on = false;
                switch_status();
            }
        }
    }
    else if (direc_car_can_go[current_direc] <= 0)
    {
        yellow_light_on = true;
    }
}

```

```

        if (on_bridge_num == 0)
        {
            yellow_light_on = false;
            switch_status();
        }
    }
}

void Bridge::switch_status()
{
    current_direc = (current_direc + 1) % 2; // 切换方向
    if (current_direc == 0)
    {
        switchCount++;
        direc_car_can_go[0] = direc_car_num[0];
        direc_car_can_go[1] = direc_car_num[1];
        direc_car_num[0] = 0;
        direc_car_num[1] = 0;
    }
    next_switch_time = stats->totalTicks + BASE_GREEN_LIGHT_TIME *
                                                                TimerTicks;
    printf("\033[1;31mnow switch the direc to %d\n\033[0m",
current_direc);
    printf("\033[1;31m%d cars to next switch\n\033[0m",
direc_car_can_go[current_direc]);
    printf("\033[1;31m0 come cars %d\t1 come cars %d\033[0m",
direc_car_can_go[0],
        direc_car_can_go[1]);
    direc_con[current_direc]->Broadcast(lock);
}
#endif

```

代码 41